# Transparent and Post-Quantum Distributed SNARK with Linear Prover Time

Zesheng Li[1,2], Xinxuan Zhang[1,2], and Yi Deng[1,2]

[1] Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China
[2] School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China
lizesheng@iie.ac.cn
zhangxinxuan@iie.ac.cn
deng@iie.ac.cn

**Abstract.** Succinct Non-interactive Arguments of Knowledge (SNARKs) allow a prover to convince a verifier of the validity of a statement using a compact proof and sublinear verification time. However, a major obstacle to the broad application of SNARKs is the high memory and computational cost required for proof generation. Distributed proof systems offer a promising solution by distributing the proving workload across multiple machines. While recent pairing-based distributed SNARKs achieve sublinear costs, they suffer from a lack of post-quantum security and transparency. Conversely, recent hash-based schemes offer these features but have been limited to quasi-linear prover time.

In this paper, we present the first fully distributed, transparent, post-quantum SNARK with a linear-time prover while maintaining polylogarithmic verification time and proof size. Our main contributions are two-fold. First, we present a distributed multivariate Polynomial IOP (PIOP) for Rank-1 Constraint Systems (R1CS) based on the Spartan framework. This is achieved by introducing a novel distributed version of the SPARK compiler, which efficiently handles the polynomial commitment scheme for sparse polynomials. Second, we propose the first transparent and post-quantum distributed polynomial commitment scheme with a linear-time prover, building upon the Brakedown framework with proof composition. By compiling our distributed polynomial commitment with both existing and newly proposed distributed PIOPs, we obtain fully distributed SNARKs for Plonkish and R1CS. Both resulting systems are transparent, post-quantum secure, and achieve linear prover time with polylogarithmic verification costs, overcoming the limitations of prior works and enhancing the scalability of zero-knowledge proof systems.

**Keywords:** Distributed SNARK · Zero-knowledge proof · Scalable

## 1 Introduction

The Succinct Non-interactive ARgument of Knowledge (SNARK) is a proof system that enables a prover to convince a verifier of the knowledge of a witness

satisfying an NP relation, while ensuring that both the proof size and the verification time are sublinear in the statement length. In recent years, SNARK constructions [11,13,1,24,6,27,2,23,8,29,4,12] have witnessed considerable advances in the theory and practical application. While SNARKs are being deployed in an expanding range of applications like ZCash[5] and zkRollup[9], their substantial memory requirements present a critical bottleneck. This is because proof generation in existing schemes incurs significant computational overhead, limiting their scalability. To overcome this limitation, a feasible solution is to construct distributed proof systems, which accelerate proof generation by distributing the overall workload to several machines.

Several recent works have focused on distributed proof systems. A prominent category includes pairing-based elliptic curve schemes such as DIZK [26], Pianist [18], Hekaton[22], HyperPianist[15], Cirrus[25], and Soloist[17]. Despite achieving sublinear verification costs and supporting general arithmetic circuits, these systems incur significant prover overhead due to computationally expensive group and pairing operations. Moreover, they lack both post-quantum security and transparency.

There are several hash-based distributed SNARKs, enabling the transparent and post-quantum property. DeVirgo[28] is the first hash-based distributed SNARK. However, it only supports data-parallel circuits. Two concurrent works [30] and [16] each introduced fully distributed SNARKs employing FRI-based polynomial commitments, utilizing distinct techniques to optimize efficiency. Furthermore, [31] presents a distributed version of BaseFold[32], constructed by compiling HyperPlonk's PIOP, resulting in a hash-based distributed SNARK with polylogarithmic communication costs. Observe that all of these schemes have quasi-linear prover time. This naturally leads to the following question:

*Can we design a transparent, post-quantum distributed SNARK with linear-time prover, polylogarithmic verifier time, and polylogarithmic proof size?*

### 1.1 Our Contribution

In this paper, we present the first fully distributed, hash-based SNARKs that achieve a linear-time prover while maintaining polylogarithmic verification time and proof size, thereby affirmatively answering the above question. Our contributions are twofold, advancing both the PIOP and the polynomial commitment scheme:

- **Distributed R1CS multivariate PIOP.** We first propose a distributed multivariate PIOP for R1CS, based on the Spartan[23] framework. As the main building block, we propose a new distributed version of SPARK polynomial commitment in Spartan, which is of independent interest.
- **Distributed hash-based polynomial commitment**.We propose the first transparent and post-quantum distributed polynomial commitment scheme that achieves linear prover time along with polylogarithmic verifier time and proof size. Our construction builds upon the Brakedown[12] framework, enhanced with proof composition.

By integrating these components, we obtain two fully distributed SNARKs:

- **Distributed SNARK for Plonkish circuits**, which is obtained by compiling our novel distributed polynomial commitment scheme with the distributed Hyperplonk PIOP from Hyperpianist [15].
- **Distributed SNARK for R1CS**, which is obtained by compiling our novel distributed polynomial commitment scheme with a new distributed multivariate PIOP for R1CS introduced in this work.

Both systems are transparent, post-quantum secure, and achieve a linear-time prover together with polylogarithmic verification time and proof size. A detailed comparison with prior work is summarized in Table 1.

**Table 1.** Comparison between our work and other hash-based distributed SNARK. Here $N$ denotes the whole circuit size, $M$ denotes the number of sub-provers, and $T := \frac{N}{M}$. $K$ denotes the "Fold-and-Batch" folding parameter in [30], where $K \in [1, T]$. "$\mathcal{P}_i$" stands for the running time of each sub-prover, "Comm" stands for the total communication costs, "$|\pi|$" denotes the proof size, and "$\mathcal{V}$" stands for the verification time."Fully" denotes whether the protocol supports general circuits. "Ours(Plonk)" denotes our distributed SNARK construction for Plonkish, "Ours(R1CS)" denotes our distributed SNARK construction for R1CS.

| **Scheme** | $\mathcal{P}_i$**(PIOP)** | $\mathcal{P}_i$**(PCS)** | **Comm.** | $\lvert\pi\rvert$&$\mathcal{V}$ | **Fully?** |
|---|---|---|---|---|---|
| deVirgo[28] | $O(T)$ | $O(T \log T)$ | $O(N)$ | $O(\log^2 N)$ | × |
| Hyperfond[31] | $O(T)$ | $O(T \log T)$ | $O(M \log^2 T)$ | $O(M \log^2 T)$ | ✓ |
| Frittata [30] | $O(T \log T)$ | $O(T \log T)$ | $O(MK + M \log T \log \frac{T}{K})$ | $O(M \log T \log \frac{T}{K} + \log^2 K)$ | ✓ |
| Li et al. [16] | $O(T)$ | $O(T \log T)$ | $O(N)$ | $O(M + \log^2 T)$ | ✓ |
| Ours(Plonk) | $O(T)$ | $O(T)$ | $O(M \log^2 T)$ | $O(M \log^2 T)$ | ✓ |
| Ours(R1CS) | $O(T)$ | $O(T)$ | $O(M^2 + M \log^2 T)$ | $O(M \log^2 T)$ | ✓ |

### 1.2 Technique Overview

We outline the high-level ideas for our distributed PIOP and distributed polynomial commitment.

**1.2.1 Distributed PIOP.** Existing PIOPs with linear prover time primarily target multivariate representations, including Spartan and Hyperplonk. Among these, Hyperplonk already offers a relatively mature distributed construction that can be adopted directly. In this paper, we focus on designing a distributed version of the Spartan PIOP.

The Spartan proof system targets the R1CS constraint system, where a circuit is represented by three matrices $A, B, C \in \mathbb{F}^{n \times m}$, where $n$ denotes the number of constraints and $m$ the number of variables. The values of $m$ and $n$ are typically comparable to the size of the circuit. All constraints can be expressed in the form $Az \circ Bz = Cz$, where $z \in \mathbb{F}^m$. In a distributed setting, a

key objective is scalability, meaning that no single sub-prover should need to store the full matrices $A, B, C$ or the vector $z$ in memory. We therefore need to design an efficient partitioning strategy that enables each prover to use minimal memory resources during the protocol.

*Distributed sumcheck in Spartan.* The Spartan protocol executes two sumcheck protocols in sequence: (1) $\sum_{x\in\{0,1\}^{\log n}}(\overline{A}(x)\cdot\overline{B}(x)-\overline{C}(x))\cdot\mathsf{eq}(x,\tau)=0$, where $\overline{P}(x)=\sum_{y\in\{0,1\}^{\log m}}\tilde{P}(x,y)\tilde{z}(y)$; and (2) $\sum_{y\in\{0,1\}^{\log m}}\tilde{P}(\mathbf{r}_x,y)\tilde{z}(y)=v_P$, for $P\in\{A,B,C\}$. Here, $\tau\in\mathbb{F}^{\log n}$ and $\mathbf{r}_x\in\mathbb{F}^{\log m}$ are random challenges generated by the verifier (the definitions of notations $\tilde{P},\tilde{z},\mathsf{eq}$ are provided in Definition 4). In a distributed setting involving $M$ sub-provers and a master prover, we employ the distributed sumcheck protocol from Hyperpianist[15], which achieves strictly linear prover time through the dynamic programming technique introduced in [27]. This leads to the following baseline approach: The matrices $A$, $B$, and $C$ are partitioned both by row and by column. Specifically, the $i$-th sub-prover holds row-based partitions $A_1^{(i)},B_1^{(i)},C_1^{(i)}\in\mathbb{F}^{\frac{n}{M}\times m}$ and column-based partitions $A_2^{(i)},B_2^{(i)},C_2^{(i)}\in\mathbb{F}^{n\times\frac{m}{M}}$. The vector $z$ is split into $z^{(0)},\ldots,z^{(M-1)}\in\mathbb{F}^{\frac{m}{M}}$. The protocol proceeds in two phases. First, the $i$-th sub-prover computes all evaluations of the partial polynomial $\overline{P}^{(i)}(x)=\sum_{y\in\{0,1\}^{\log m}}\tilde{P}_1^{(i)}(x,y)\tilde{z}(y)$ over $x\in\{0,1\}^{\log n-\log M}$, for $P\in\{A,B,C\}$. The provers and verifier then run the sumcheck protocol (1). Second, the $i$-th sub-prover computes all evaluations of the partial polynomial $\tilde{P}^{(i)}(\mathbf{r}_x,y)=\sum_{x\in\{0,1\}^{\log n}}\tilde{P}_2^{(i)}(x,y)\cdot\mathsf{eq}(\mathbf{r}_x,x)$ over $y\in\{0,1\}^{\log m-\log M}$, for $P\in\{A,B,C\}$. The provers and verifier then run the sumcheck protocol (2). However, this initial scheme presents several challenges. In the following, we elaborate on these issues and our solutions.

The first challenge arises before the execution of the sumcheck protocol (1). The $i$-th sub-prover must possess all evaluations of $\overline{A}^{(i)}(x),\overline{B}^{(i)}(x),\overline{C}^{(i)}(x)$ over $x\in\{0,1\}^{\log n-\log M}$. Computing these values requires that a single sub-prover holds the entire vector $z$, whose length is nearly $O(m)$. This violates the principle of scalability. If the vector $z$ is initially distributed among all provers, the protocol would inevitably incur linear communication overhead during execution, which becomes prohibitive for large-scale circuits.

To address this challenge, we leverage the sparsity of matrices $A, B, C$, each containing only $O(n)$ non-zero entries. We illustrate the approach using matrix $A$, as the reasoning for $B$ and $C$ is analogous. Crucially, we observe that the $i$-th prover only requires those entries of vector $z$ that correspond to the column indices of the non-zero entries in its assigned sub-matrix $A_1^{(i)}$ to compute $\overline{A}^{(i)}(x)$. As a result, the number of entries of $z$ the $i$-th sub-prover needs to store is almost equal to the non-zero entries of $A_1^{(i)}$. Since $A$ is partitioned row-wise into M roughly equal fragments, the average number of non-zero entries per sub-matrix $A_1^{(i)}$ is $O(\frac{n}{M})$. This approach substantially reduces memory overhead: the average memory required per sub-prover is now $O(\frac{n}{M})$, a significant improvement over the $O(m)$ requirement in the initial scheme.

The second challenge arises before the execution of the sumcheck protocol(2). The $i$-th sub-prover must compute the evaluations of $\tilde{A}^{(i)}(\mathbf{r}_x, y), \tilde{B}^{(i)}(\mathbf{r}_x, y)$ and $\tilde{C}^{(i)}(\mathbf{r}_x, y)$ over $y \in \{0,1\}^{\log m - \log M}$, where $\tilde{P}^{(i)}(\mathbf{r}_x, y) = \sum_{t \in \{0,1\}^{\log n}} \mathsf{eq}(\mathbf{r}_x, t) \cdot \tilde{P}_2^{(i)}(t, y)$, for $P \in \{A, B, C\}$. This computation requires $O(\frac{n \log n}{M})$ field operations per sub-prover, as each evaluation of the function $\mathsf{eq}$ takes $O(\log n)$ time. However, to maintain an overall linear prover time, every sub-prover must store *all* evaluations of the polynomial $\mathbf{e}(t) = \mathsf{eq}(\mathbf{r}_x, t)$ over $\{0,1\}^{\log n}$, resulting in a memory cost of $O(n)$. This memory requirement again violates the scalability principle of the distributed system.

To address this issue, we observe that the vector $\mathbf{e}$ can be decomposed into a tensor product of two smaller vectors $\mathbf{e}_1$ and $\mathbf{e}_2$, where $\mathbf{e}_1 = \otimes_{i=1}^{\log n - \log M}(1 - \mathbf{r}_x[i], \mathbf{r}_x[i])$ and $\mathbf{e}_2 = \otimes_{i=1}^{\log M}(1 - \mathbf{r}_x[i], \mathbf{r}_x[i])$. Each sub-prover only needs to generate and store $\mathbf{e}_1$, while the master prover generates $\mathbf{e}_2$ and broadcasts it to all sub-provers. Each sub-prover can then reconstruct the required value of $\mathbf{e}(t)$ by looking up and multiplying the corresponding entries from $\mathbf{e}_1$ and $\mathbf{e}_2$. Each sub-prover only requires $O(\frac{N}{M})$ time to generate $\mathbf{e}_1$. Although this method introduces a communication cost of $O(M^2)$ due to the master prover sending $\mathbf{e}_2$ to every sub-prover, it reduces the memory requirement per sub-prover from $O(N)$ to $O(\frac{N}{M} + M)$, thereby satisfying the scalability requirement.

*Distributed Spark Compiler* An essential component of Spartan[23] is Spark, which efficiently handles commitments for sparse multilinear polynomials. Specifically, at the end of the Spartan protocol, the prover is required to convince the verifier of the evaluations $\tilde{A}(\mathbf{r}_x, \mathbf{r}_y) = v_A, \tilde{B}(\mathbf{r}_x, \mathbf{r}_y) = v_B, \tilde{C}(\mathbf{r}_x, \mathbf{r}_y) = v_C$. A naive approach to verifying these relations would lead to quadratic prover time. To avoid this, the Spark compiler applies offline memory checking techniques from [3], reducing the verification to the following set-equality relation:

$$\{Init(\mathbf{u})\}_{\mathbf{u} \in \{0,1\}^m} \cup \{WS(\mathbf{v})\}_{\mathbf{v} \in \{0,1\}^n} = \{Audit(\mathbf{u})\}_{\mathbf{u} \in \{0,1\}^m} \cup \{RS(\mathbf{v})\}_{\mathbf{v} \in \{0,1\}^n}$$

where $Init, Audit \in \mathbb{F}^{\log m}$, and $WS, RS \in \mathbb{F}^{\log n}$ are generated by the prover. While analogous to the multiset check in HyperPlonk [7], this relation differs in involving two variables $\mathbf{u}, \mathbf{v}$ of distinct lengths. This difference prevents the direct application of existing methods(in [15,31]) for constructing a distributed version. We therefore formally define it as a *multiset equality check*. We utilize the logarithmic derivatives technique in [14] to transform the multiset equality check into an identity involving sums of rational functions:

$$\sum_{\mathbf{u} \in \{0,1\}^m} \frac{1}{\gamma + Init(\mathbf{u})} + \sum_{\mathbf{v} \in \{0,1\}^n} \frac{1}{\gamma + WS(\mathbf{v})} = \sum_{\mathbf{u} \in \{0,1\}^m} \frac{1}{\gamma + Audit(\mathbf{u})} + \sum_{\mathbf{v} \in \{0,1\}^n} \frac{1}{\gamma + RS(\mathbf{v})} \quad (1)$$

We then show that Equation 1 can indeed be reduced to a distributed rational sumcheck by means of algebraic rearrangement and combination over a

common denominator. Specifically, Equation 1 can be transformed into the following identity:

$$\sum_{\mathbf{u}\in\{0,1\}^m} \frac{Audit(\mathbf{u}) - Init(\mathbf{u})}{(\gamma + Audit(\mathbf{u}))(\gamma + Init(\mathbf{u}))} = \sum_{\mathbf{v}\in\{0,1\}^m} \frac{WS(\mathbf{v}) - RS(\mathbf{v})}{(\gamma + WS(\mathbf{v}))(\gamma + RS(\mathbf{v}))} \tag{2}$$

Finally, the distributed rational sumcheck protocol applies directly to the verification of Equation 2.

By integrating the distributed sumcheck in Spartan with the distributed Spark compiler, we construct the complete distributed Spartan PIOP. The detailed procedure is presented in Section 3.

**1.2.2 Distributed PCS.** We now provide an overview of the techniques used in our distributed polynomial commitment scheme. Our starting point is the Brakedown[12] polynomial commitment, which is transparent, post-quantum secure, and features a linear-time prover. Brakedown employs a linear-time encoding scheme $\mathsf{Enc}$. To prove the evaluation $f(\mathbf{x}_1, \mathbf{x}_2) = v$ for a $\mu$-variate multilinear polynomial $f$, its evaluations are arranged into a matrix $M_f \in \mathbb{F}^{n_1 \times n_2}$, where $n_1 n_2 = 2^\mu = N$. In the commit phase, the prover encodes each row of $M_f$ to obtain $M'_f \in \mathbb{F}^{n_1 \times cn_2}$ and commits to $M'_f$ via a Merkle tree. The evaluation phase consists of a proximity test and a consistency check. In the proximity test, the verifier sends a random vector $\mathbf{a} \in \mathbb{F}^{n_1}$, and the prover responds with the linear combination vectors $F_1 = \mathbf{a}^T M_f$ and $F_2 = \mathbf{e}_{row}^T M_f$, where $\mathbf{e}_{row} = \otimes_{i=0}^{n_1-1}(1 - \mathbf{x}_1[i], \mathbf{x}_1[i])$. The verifier samples a random index set $\mathcal{I}$; the prover returns the corresponding columns $R$ of $M'_f$ with Merkle proofs. The verifier checks the proofs and that $E_1[i] = \sum_{j\in[n_2]} \mathbf{a}[j]R[j,i]$ , for each $i \in [\ell]$, where $E_1 = \mathsf{Enc}(F_1)$. The consistency check almost identical to the proximity one but with the difference that the prover sends $F_2 = \mathbf{e}_{col}^T M_f$, where $\mathbf{e}_{col} = \otimes_{i=0}^{n_2-1}(1 - \mathbf{x}_1[i], \mathbf{x}_1[i])$.

Subsequent work [20] and [21] optimizes the scheme via proof composition: $f$ is arranged into a wider matrix $M_f \in \mathbb{F}^{B\times \frac{N}{B}}$ and the linearly combined vectors $E_1, E_2, F_1, F_2$ are committed. The verification of $E_1 = \mathsf{Enc}(F_1)$ and $E_2 = \mathsf{Enc}(F_2)$ is embedded into an arithmetic circuit and proven using the sumcheck protocol, reducing proof size and verification time($O(\log^2 N)$ for [20] and $O(\frac{N}{B} + \log^2 B)$ for [21]). Details are provided in Subsection 4.1.

When we come to consider the distributed setting involving $M$ sub-provers and a master prover, two main challenges arise: (1) how to generate commitments to the linearly combined vectors $E_1, E_2, F_1, F_2$, and (2) how to efficiently prove relations expressed as arithmetic circuits, i.e. $E_1 = \mathsf{Enc}(F_1), E_2 = \mathsf{Enc}(F_2)$. We outline how to address these two challenges.

*Commit to linearly combined vectors.* In the Brakedown protocol with proof composition, the prover commits to $F_1, F_2$, as well as to their respective encoding vector $E_1 = \mathsf{Enc}(F_1), E_2 = \mathsf{Enc}(F_2)$. We illustrate the process using $F_1$ as an example; the procedures for $F_2, E_1$ and $E_2$ are similar to $F_1$.

The matrix $M_f$ is partitioned row-wise into $M$ sub-matrices $M_f^{(0)}, \ldots, M_f^{(M-1)} \in \mathbb{F}^{\frac{B}{M} \times \frac{N}{B}}$, and the randomly sampled vector $\mathbf{a} \in \mathbb{F}^B$ is divided into $M$ segments $\mathbf{a}^{(0)}, \ldots, \mathbf{a}^{(M-1)} \in \mathbb{F}^{\frac{B}{M}}$. The $i$-th sub-prover, holding $M_f^{(i)}$ and $\mathbf{a}^{(i)}$, computes the partial sum $F_1^{(i)} = \sum_{k \in \{0,\ldots,\frac{N}{B}-1\}} \mathbf{a}^{(i)}[k] M_f^{(i)}[k]$, such that the full vector is obtained as $F_1 = \sum_{i \in \{0,\ldots,M-1\}} F_1^{(i)}$. We demonstrate that for a linearly combined polynomial $\sum_i a_i f_i$, it suffices to commit to and open each $f_i$ individually using an arbitrary polynomial commitment scheme. In the distributed setting, each $f_i$ is committed and opened by the $i$-th sub-prover. In the case of committing to $F_1$ in a distributed manner, we have $f_i = F_1^{(i)}$ and $a_i = 1$ for all $i$. For the Merkle-tree hash commitments, each sub-prover generates a commitment and a corresponding opening proof for the matrix or vector it holds. The verifier checks each proof individually, and the overall verification succeeds only if every proof is valid.

When $B = M \log \frac{N}{M}$, the prover time can achieve linear complexity. Compared to the underlying polynomial commitment, our method introduces an overhead of $O(M)$ factor in proof size and verification time.

For more details, please refer to Subsection 4.2.

*Proving the circuit relation* The core task is to prove that $E_1, E_2 \in \mathbb{F}^{cl}$ are valid encodings of $F_1, F_2 \in \mathbb{F}^l$ under a systematic code with rate $1/c$. For simplicity, we explain how to prove the encoding algorithm $E = Enc(F)$. This requires establishing two conditions: (1) Codeword validity: $E \cdot H = 0$, where $H$ is a sparse parity-check matrix; and (2) Systematic consistency: the first $l$ entries of $E$ are equal to $F$. As proven in [20], the parity-check matrix used in Brakedown's linear code is sparse; therefore, $H$ is sparse.

For condition (1), we express the check as a sumcheck relation $\sum \tilde{E}(b) \cdot \tilde{H}_u(b) = 0$, where $\tilde{H}_{\mathbf{u}}(b) = \tilde{H}(b, \mathbf{u})$. When we attempt to apply the distributed sumcheck protocol in Hyperpianist[15], an issue arises because $i$-th prover holds the $\tilde{E}_1^{(i)}$ such that $\tilde{E} = \sum_{i \in \{0,\ldots M-1\}} \tilde{E}^{(i)}$, rather than the $i$-th partial polynomial of $\tilde{E}$ as required by the distributed sumcheck protocol in Hyperpianist[15]. A key observation is that every sub-prover $\mathcal{P}_i$ holds the parity-check matrix $H$. This is feasible because $H$ is sparse, containing only $O(\frac{cN}{B})$ non-zero entries. Consequently, each sub-prover can locally compute the evaluations of $\tilde{H}_u$ efficiently. In the $k$-th round of sumcheck, the $i$-th sub-prover computes $\tilde{E}_k^{(i)}(X_k, x) = \sum_{x \in \{0,1\}^{c\ell-k}} \tilde{E}_1^{(i)}(r_1, \ldots, r_{k-1}, X_k, x), \tilde{H}_{\mathbf{u},k}(X_k, x) := \tilde{H}_{\mathbf{u}}(r_1, \ldots, r_{k-1}, X_k, x)$, and then $Q_k^{(i)}(X_k) \leftarrow \sum_{x \in \{0,1\}^{\log \frac{cN}{B}-k}} \tilde{E}_k^{(i)}(X_k, x) \cdot \tilde{H}_{\mathbf{u},k}(X_k, x)$. The master prover $\mathcal{P}_0$ then aggregates the partial results from sub-provers to $Q_k(X_k)$.At the final stage, the "query" for $\tilde{E}$ is handled by our distributed polynomial commitment scheme, while the query for $\tilde{H}_u$ is handled using the distributed BaseFold protocol.

For condition (2), it is enough to prove that $\tilde{E}_1(0, x) = \tilde{F}_1(x)$ for any $x \in \{0,1\}^{\log \ell}$. This reduces to querying the values of $\tilde{E}$ and $\tilde{F}$ at specific points. These queries are instantiated directly using the evaluation phases of our dis-

tributed polynomial commitment scheme, ensuring linear prover time and poly-logarithmic verification costs.

For more details, please refer to Subsection 4.3.

## 2 Preliminary

**Notations** Let $\lambda$ denote the security parameter. A function $f(\lambda)$ is *negligible* (denoted by $\mathsf{negl}(\lambda)$) if $\forall c > 0$, $f(\lambda) = o(\lambda^{-c})$. For $n \in \mathbb{N}$, define $[n] := \{0, 1, \ldots, n-1\}$, and $[a : b] = \{a, a+1, \ldots, b\}$. Let $\mathbb{F} = \mathbb{F}_p$ be a finite field of prime order $p$, where $|\mathbb{F}| = \Omega(2^\lambda)$. $\mathcal{F}_\mu^{(\leq d)}(X)$ denotes the set of $\mu$-variate polynomials of degree at most $d$ over $\mathbb{F}$.

Let $v$ be a vector and $M$ a matrix. Denote by $v[i]$ the $i$-th entry of $v$, and by $M[i, j]$ the entry in the $i$-th row and $j$-th column of $M$. Let $M[i, :]$ represent the row vector corresponding to the $i$-th row of $M$, and $M[:, j]$the column vector corresponding to the $j$-th column of $M$.

Let $bin : \mathbb{N} \to \{0,1\}^l$ be a function that outputs the binary representation of a number, i.e.,$\mathbf{b} = bin(c)$ satisfies $c = \sum_{i \in [l]} 2^l \cdot \mathbf{b}[l]$, and let $bin^{-1} : \{0,1\}^l \to \mathbb{N}$ denote its inverse mapping.

**Definition 1 (Argument of Knowledge[7]).** *An interactive protocol $\Pi = (\mathsf{Setup}, \mathcal{I}, \mathcal{P}, \mathcal{V})$ is an argument of knowledge for an indexed relation $\mathcal{R}$ with knowledge error $\delta : \mathbb{N} \to [0,1]$ if the following properties hold, where given an index $\mathtt{i}$, common input $\mathtt{x}$ and prover witness $\mathtt{w}$, the deterministic indexer outputs $(\mathsf{vk}, \mathsf{pk}) \leftarrow \mathcal{I}(\mathtt{i})$ and the output of the verifier is denoted by the random variable $\langle \mathcal{P}(\mathsf{pk}, \mathtt{x}, \mathtt{w}), \mathcal{V}(\mathsf{vk}, \mathtt{x}) \rangle$:*

- ***Completeness**: For all $(\mathtt{i}, \mathtt{x}, \mathtt{w}) \in \mathcal{R}$*

$$\Pr\left[\langle \mathcal{P}(\mathsf{pk}, \mathtt{x}, \mathtt{w}), \mathcal{V}(\mathsf{vk}, \mathtt{x}) \rangle = 1 \;\middle|\; \begin{array}{c} \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda) \\ (\mathsf{vk}, \mathsf{pk}) \leftarrow \mathcal{I}(\mathsf{pp}, \mathtt{i}) \end{array}\right] = 1.$$

- *$\delta$-**Knowledge Soundness**: There exists a PPT extractor $\mathcal{E}$ such that given oracle access to any pair of PPT adversarial algorithms $(\mathcal{A}_1, \mathcal{A}_2)$, the following holds:*

$$\Pr\left[\begin{array}{c} \langle \mathcal{A}_2(\mathtt{i}, \mathtt{x}, \mathsf{st}), \mathcal{V}(\mathsf{vk}, \mathtt{x}) \rangle = 1 \\ \wedge \\ (\mathtt{i}, \mathtt{x}, \mathtt{w}) \notin \mathcal{R} \end{array} \;\middle|\; \begin{array}{c} \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda) \\ (\mathtt{i}, \mathtt{x}, \mathsf{st}) \leftarrow \mathcal{A}_1(\mathsf{pp}) \\ (\mathsf{vk}, \mathsf{pk}) \leftarrow \mathcal{I}(\mathsf{pp}, \mathtt{i}) \\ \mathtt{w} \leftarrow \mathcal{E}^{\mathcal{A}_1, \mathcal{A}_2}(\mathsf{pp}, \mathtt{i}, \mathtt{x}) \end{array}\right] \leq \delta(|\mathtt{i}| + |\mathtt{x}|).$$

  *An interactive protocol has **knowledge soundness** property if the knowledge error $\delta$ is negligible in $\lambda$.*

*An interactive argument of knowledge protocol can be made non-interactive by the Fiat-Shamir transformation [10] if it is public coin.*

- ***Public coin**: An interactive protocol is public-coin if $\mathcal{V}$'s messages are chosen uniformly at random.*

*If the protocol further satisfies succinctness, then it is a Succinct Non-interactive Argument of Knowledge(SNARK).*

- ***Succinctness**: The proof size is $|\pi| = \mathsf{poly}(\lambda, \log|\mathcal{C}|)$ and the verification time is $\mathsf{poly}(\lambda, \log|\mathcal{C}|)$, where $\mathcal{C}$ is the arithmetic circuit representing the relation $\mathcal{R}$.*

**Definition 2 (Polynomial Commitment[32]).** *A polynomial commitment scheme is a tuple of PPT algorithms* $(\mathsf{Gen}, \mathsf{Commit}, \mathsf{Open}, \mathsf{Eval})$ *where:*

- $\mathsf{Gen}(1^\lambda) \to \mathsf{pp}$ *takes the security parameters* $\lambda$*, and output the public parameter* $\mathsf{pp}$*.*
- $\mathsf{Commit}(\mathsf{pp}, f) \to \mathsf{com}_f$ *takes the public parameters* $\mathsf{pp}$ *and a multilinear polynomial* $f$*, outputs a public commitment* $\mathsf{com}_f$*;*
- $\mathsf{Open}(\mathsf{pp}, \mathsf{com}_f, f) \to b$ *takes the public parameters* $\mathsf{pp}$ *and the public commitment* $\mathsf{com}_f$*, outputs a bit* $b$*;*
- $\mathsf{Eval}(\mathsf{pp}, \mathsf{com}_f, \boldsymbol{x}, y; f) \to b$ *is a protocol between the prover* $\mathcal{P}$ *and verifier* $\mathcal{V}$ *with public parameters* $\mathsf{pp}$*, the commitment* $\mathsf{com}_f$*, an evaluation point* $\boldsymbol{x} \in \mathbb{F}^\mu$*, the value* $y$ *and the multilinear polynomial* $f$*.* $\mathcal{P}$ *secretly holds a multilinear* $f$ *and wants to convince* $\mathcal{V}$ *that* $\mathsf{com}_f$ *is a commitment to* $f$ *and* $f(\boldsymbol{x}) = y$*. At the end of protocol,* $\mathcal{V}$ *outputs a bit* $b$*.*

*A polynomial commitment satisfies the following properties:*

- ***Completeness.** For any $\mu \in \mathbb{N}$, polynomial $f \in \mathcal{F}_\mu^{(\leq 1)}$ and $\boldsymbol{x} \in \mathbb{F}^\mu$,*

$$Pr\left[\mathsf{Eval}(\mathsf{pp}, \mathsf{com}_f, \boldsymbol{x}, y; f) = 1 \,\middle|\, \begin{matrix} \mathsf{pp} \leftarrow \mathsf{Gen}(1^\lambda, \mu) \\ \mathsf{com}_f \leftarrow \mathsf{Commit}(\mathsf{pp}, f) \end{matrix}\right] = 1$$

- ***Binding.** For any $\mu \in \mathbb{N}$, and any PPT adversary $\mathcal{A}$,*

$$Pr\left[b_0 = b_1 = 1 \wedge f_0 \neq f_1 \,\middle|\, \begin{matrix} \mathsf{pp} \leftarrow \mathsf{Gen}(1^\lambda, \mu) \\ (\mathsf{com}, f_0, f_1) \leftarrow \mathcal{A}(\mathsf{pp}) \\ b_0 \leftarrow \mathsf{Open}(\mathsf{pp}, \mathsf{com}, f_0) \\ b_1 \leftarrow \mathsf{Open}(\mathsf{pp}, \mathsf{com}, f_1) \end{matrix}\right] = 1$$

- ***Knowledge soundness.** For any PPT adversary $\mathcal{P}^*$, there exists a PPT extractor $\mathcal{E}$ with access to $\mathcal{P}^*$'s messages during the protocol,*

$$Pr\left[\begin{matrix} \mathsf{pp} \leftarrow \mathsf{Gen}(1^\lambda) \\ (y^*, \boldsymbol{x}^*, \mathsf{com}^*) \leftarrow \mathcal{P}^*(\mathsf{pp}) \\ f^* \leftarrow \mathcal{E}^{\mathcal{P}^*(\cdot)}(\mathsf{pp}) \\ \mathsf{Eval}(\mathsf{pp}, \mathsf{com}^*, \boldsymbol{x}^*, y^*; f^*) = 1 \end{matrix}\right] \approx Pr\left[\begin{matrix} \mathsf{pp} \leftarrow \mathsf{Gen}(1^\lambda) \\ (y^*, \boldsymbol{x}^*, \mathsf{com}^*) \leftarrow \mathcal{P}^*(\mathsf{pp}) \\ f^* \leftarrow \mathcal{E}^{\mathcal{P}^*(\cdot)}(\mathsf{pp}) \\ ((\mathsf{com}^*, \boldsymbol{x}^*, y^*); f^*) \in \mathcal{R}_{Eval} \end{matrix}\right]$$

*where* $\mathcal{R}_{Eval} = \{((\mathsf{com}^*, \boldsymbol{x}^*, y^*); f) | f(\mathbf{z}) = y \wedge \mathsf{Open}(\mathsf{pp}, C, f) = 1\}$*.*

**Definition 3 (Polynomial Interactive Oracle Proof [7]).** *A public-coin polynomial interactive oracle proof (PIOP) is a public-coin interactive proof for a polynomial oracle relation* $\mathcal{R} = (\mathtt{i}; \mathtt{x}; \mathtt{w})$*, where* $\mathtt{i}$ *and* $\mathtt{x}$ *can contain oracles to* $n$*-variate polynomials over some field* $\mathbb{F}$*. These oracles can be queried at arbitrary points in* $\mathbb{F}^n$ *to evaluate the polynomial at these points. We denote an oracle to a polynomial* $f$ *by* $[[f]]$*.*

**Definition 4 (Multilinear Extension).** *For any $f : \{0,1\}^\mu \to \mathbb{F}$, there exists a unique multilinear polynomial $\tilde{f} : \mathbb{F}^\mu \to \mathbb{F}$ such that for all $\boldsymbol{x} \in \{0,1\}^\mu$, $\tilde{f}(\boldsymbol{x}) = f(\boldsymbol{x})$. The polynomial $\tilde{f}$ is called the multilinear extension(MLE) of $f$, and can be expressed as $\tilde{f}(\boldsymbol{X}) = \sum_{\boldsymbol{x}\in\{0,1\}^\mu} f(\boldsymbol{x}) \cdot \mathsf{eq}(\boldsymbol{x}, \boldsymbol{X})$, where $\mathsf{eq}(\boldsymbol{x}, \boldsymbol{X}) := \prod_{i=1}^{\mu}(\boldsymbol{x}[i]\boldsymbol{X}[i] + (1-\boldsymbol{x}[i])(1-\boldsymbol{X}[i]))$.*

*We generalize the definition of MLE to vectors and matrices. For a vector $v \in \mathbb{F}^\ell$, the MLE of $v$ is the multilinear polynomial $\tilde{v} : \mathbb{F}^\ell \to \mathbb{F}$ such that for all $\boldsymbol{x} \in \{0,1\}^{\log \ell}$, $\tilde{v}(\boldsymbol{x}) = v[bin^{-1}(\boldsymbol{x})]$. For the matrix $M \in \mathbb{F}^{m\times n}$, the MLE of $M$ is the multilinear polynomial $\tilde{M} : \mathbb{F}^{\log m + \log n} \to \mathbb{F}$ such that for all $\boldsymbol{x} \in \{0,1\}^{\log m}$ and $\boldsymbol{y} \in \{0,1\}^{\log n}$, $\tilde{M}(\boldsymbol{x}, \boldsymbol{y}) = M[bin^{-1}(\boldsymbol{x}), bin^{-1}(\boldsymbol{y})]$.*

**Definition 5 (Sumcheck Relation for High Degree Polynomials [19,7]).** *The sumcheck relation for high-degree multi-variate polynomials $\mathcal{R}_{SUM}$ is the set of tuples $(\mathbb{x}; \mathbb{w}) = ((v, [[f]]); f)$, where $f \in \mathcal{F}_\mu^{(\leq d)}$, and $\sum_{x\in\{0,1\}^\mu} f(x) = v$.*

The sumcheck protocol has been shown to satisfy completeness and knowledge soundness, as established in [7]. Furthermore, a distributed variant of the sumcheck protocol is introduced in [15]. Additional details regarding the sumcheck protocol are provided in Appendix A.

**Definition 6 (R1CS [11,23]).** *Consider a finite field $\mathbb{F}$. Let the public parameters consist of size bounds $m, n, N, \ell \in \mathbb{N}$ where $n > \ell$. The R1CS structure consists of sparse matrices $A, B, C \in \mathbb{F}^{n\times n}$ with at most $m$ non-zero entries in each matrix. A R1CS instance $x \in \mathbb{F}^\ell$ consists of public inputs and outputs and is satisfied by a witness $W \in \mathbb{F}^{n-\ell-1}$ if $(A \cdot Z) \circ (B \cdot Z) = C \cdot Z$, where $Z = (W, x, 1)$. We assume that $m = O(n)$ throughout the paper.*

## 3 Distributed PIOP for R1CS

In this section, we present a distributed polynomial IOP (PIOP) for R1CS, building upon the Spartan[23] framework. We begin by recalling the construction of the distributed rational sumcheck PIOP presented in [15], which serves as an underlying building block for our Spartan PIOP(Subsection 3.1). Then we introduce a multiset equality check relation and describe its distributed proof procedure (Subsection 3.2). It is then used to construct a distributed variant of Spark (Subsection 3.3), which serves as a key building block of our distributed Spartan PIOP(Subsection 3.4). Finally, we analyze the security and complexity in Subsection 3.5.

### 3.1 Distributed Rational Sumcheck PIOP

We begin by recalling the definition of rational sumcheck:

**Definition 7 (Rational Sumcheck Relation [15]).** *The relation $\mathcal{R}_{RSUM}$ is the set of all tuples $(\mathbb{x}; \mathbb{w}) = ((v, [\![p]\!], [\![q]\!]); (p, q))$, where $p, q \in \mathcal{F}_n^{(\leq d)}$, $q(x) \neq 0$ for all $x \in \{0,1\}^n$ and $\sum_{x\in\{0,1\}^n} \frac{p(x)}{q(x)} = v$.*

We show how we prove rational sumcheck relation $\sum_{x\in\{0,1\}^n} \frac{p(x)}{q(x)} = v$ in distributed setting. The main idea is to find the multilinear interpolation of the denominator part $f(x) = \frac{1}{q(x)}$. Thus it is enough to prove: (1) $f(x)\cdot q(x) - 1 = 0$ for all $x \in \{0,1\}^n$ , (2) $\sum_{x\in\{0,1\}^n} p(x)f(x) = v$.

Condition (1) corresponds to zerocheck relation, as defined in Definition 8.

**Definition 8 (Zerocheck Relation).** *The relation $\mathcal{R}_{ZERO}$ is the set of all tuples $(\mathbb{x};\mathbb{w}) = (([[f]]); f)$ where $f \in \mathcal{F}_n^{(\leq d)}$ and $f(x) = 0$ for all $x \in \{0,1\}^n$.*

The zerocheck relation can be reduced to the sumcheck relation (Definition 5). We show the detailed construction of the distributed zerocheck PIOP in Protocol 1.

Condition (2) corresponds to the sumcheck relation (Definition 5). Therefore, the distributed sumcheck PIOP(Protocol A.2) can be directly applied to prove it in a distributed setting.

We present the construction for the distributed rational sumcheck PIOP in Protocol 2.

**Protocol 1 Distributed ZeroCheck PIOP**.
$\mathcal{P}_0$ claims $([[f]]; f) \in \mathcal{R}_{ZERO}$ to $\mathcal{V}$. $\mathcal{P}_i$ holds $f^{(i)}(x) = f(x, bin(i)), i \in [M]$.

1. $\mathcal{V}$ samples $r \leftarrow \mathbb{F}^n$ and sends it to $\mathcal{P}_0$. $\mathcal{P}_0$ transmits $r$ to the other $\mathcal{P}_i$.
2. Each $\mathcal{P}_i$ views $r$ as $r = (r', r'') \in \mathbb{F}^{n-m} \times \mathbb{F}^m$, and locally computes $\hat{f}^{(i)}(x) = f^{(i)}(x) \cdot \widetilde{eq}(x, r') \cdot \widetilde{eq}(bin(i), r'')$.
3. $\mathcal{P}_0, \cdots, \mathcal{P}_{M-1}$ and $\mathcal{V}$ run the distributed Sumcheck PIOP (Protocol A.2) to check $((0, [[\hat{f}]]); \hat{f}) \in \mathcal{R}_{SUM}$.

**Protocol 2 Distributed Rational Sumcheck PIOP**.
$\mathcal{P}_0$ claims $((v, [[p]], [[q]]); (p, q)) \in \mathcal{R}_{RSUM}$ to $\mathcal{V}$. $\mathcal{P}_i$ holds $p^{(i)}(x) = p(x, bin(i))$, $q^{(i)}(x) = q(x, bin(i))$, for $i \in [M]$.

1. Each $\mathcal{P}_i$ computes $f^{(i)}(x) = \frac{1}{q^{(i)}(x)}$, $\forall x \in \{0,1\}^{n-m}$.
2. $\mathcal{P}_0$ sends an oracle $[[f]]$ to $\mathcal{V}$.
3. $\mathcal{P}_0, \cdots, \mathcal{P}_{M-1}$ and $\mathcal{V}$ run the distributed ZeroCheck PIOP (Protocol 1)to check $(([[f \cdot q - 1]]); f \cdot q - 1) \in \mathcal{R}_{ZERO}$.
4. $\mathcal{P}_0, \cdots, \mathcal{P}_{M-1}$ and $\mathcal{V}$ run the distributed SumCheck PIOP (Protocol A.2) to check $((v, [[p \cdot f]]); p \cdot f) \in \mathcal{R}_{RSUM}$.

### 3.2 Distributed multiset Equality Check PIOP

We define the multiset equality check relation as follows:

**Definition 9 (multiset equality check).** *The relation $\mathcal{R}_{Muleq}$ is the set of all tuples $(\mathbb{x}, \mathbb{w}) = (([[f_1]], [[f_2]], [[g_1]], [[g_2]], \mu, \nu); (f_1, f_2, g_1, g_2))$ where $f_1, g_1 \in \mathcal{F}_\mu^{(\leq 1)}$ and $f_2, g_2 \in \mathcal{F}_\nu^{(\leq 1)}$, such that:*

$$\{f_1(x)\}_{x\in\{0,1\}^\mu} \cup \{f_2(x)\}_{x\in\{0,1\}^\nu} = \{g_1(x)\}_{x\in\{0,1\}^\mu} \cup \{g_2(x)\}_{x\in\{0,1\}^\nu}$$

The multiset equality check can be reduced to verifying the equality of two univariate polynomials $F(X) = \Pi_{t\in\{0,1\}^\mu}(X + f_1(t))\Pi_{t\in\{0,1\}^\nu}(X + f_2(t))$, and $G(X) = \Pi_{t\in\{0,1\}^\mu}(X+g_1(t))\Pi_{t\in\{0,1\}^\nu}(X+g_2(t))$. We then apply the logarithmic derivatives technique from [14] to this polynomial identity test, as formalized by the following lemma.

**Lemma 1.** *[14] Let $\{a_i\}_{i\in[n]}$ and $\{b_i\}_{i\in[n]}$ be sequences over a field $\mathbb{F}$ with characteristic $|\mathbb{F}| > n$. Then $\Pi_{i\in[n]}(X + a_i) = \Pi_{i\in[n]}(X + b_i)$ in $F[X]$ if and only if $\sum_{i\in[n]} \frac{1}{X+a_i} = \sum_{i\in[n]} \frac{1}{X+b_i}$ in the rational function field $F(X)$.*

Thus, to efficiently verify whether $F(X) \equiv G(X)$, it suffices to check the following identity of rational functions:

$$\sum_{t\in\{0,1\}^\mu} \frac{1}{X + f_1(t)} + \sum_{t\in\{0,1\}^\nu} \frac{1}{X + f_2(t)} \equiv \sum_{t\in\{0,1\}^\mu} \frac{1}{X + g_1(t)} + \sum_{t\in\{0,1\}^\nu} \frac{1}{X + g_2(t)} \tag{3}$$

This is equivalent to:

$$\sum_{t\in\{0,1\}^\mu} \left(\frac{1}{X + f_1(t)} - \frac{1}{X + g_1(t)}\right) \equiv \sum_{t\in\{0,1\}^\nu} \left(\frac{1}{X + g_2(t)} - \frac{1}{X + f_2(t)}\right) \tag{4}$$

By sampling a random challenge $\gamma_2$, the identity reduces to an equality on the evaluation point:

$$\sum_{t\in\{0,1\}^\mu} \left(\frac{1}{\gamma_2 + f_1(t)} - \frac{1}{\gamma_2 + g_1(t)}\right) = \sum_{t\in\{0,1\}^\nu} \left(\frac{1}{\gamma_2 + g_2(t)} - \frac{1}{\gamma_2 + f_2(t)}\right) \tag{5}$$

Combining the terms over a common denominator yields the concrete verification equation:

$$\sum_{t\in\{0,1\}^\mu} \frac{g_1(t) - f_1(t)}{(\gamma_2 + f_1(t))(\gamma_2 + g_1(t))} = \sum_{t\in\{0,1\}^\nu} \frac{f_2(t) - g_2(t)}{(\gamma_2 + f_2(t))(\gamma_2 + g_2(t))} \tag{6}$$

To prove the above equation, the prover sends $s \leftarrow \sum_{t\in\{0,1\}^n}(\frac{1}{\gamma_2+f_1(t)} - \frac{1}{\gamma_2+g_1(t)})$ to the verifier and claims that $s$ equals both sides of the equation. In the distributed version, $\mathcal{P}_i$ computes $s^{(i)} \leftarrow \sum_{t\in\{0,1\}^{n-\log M}}(\frac{1}{\gamma_2+f_1^{(i)}(t)} - \frac{1}{\gamma_2+g_1^{(i)}(t)})$ and $\mathcal{P}_0$ sums $s^{(i)}$ up to $s$. Finally, $\mathcal{P}_0, \ldots, \mathcal{P}_{M-1}$ and $\mathcal{V}$ run distributed rational sumcheck protocol[15](Protocol 2) for the relation $\sum_{t\in\{0,1\}^\mu} \frac{g_1(t)-f_1(t)}{(\gamma_2+f_1(t))(\gamma_2+g_1(t))} = s$ and $\sum_{t\in\{0,1\}^\nu} \frac{f_2(t)-g_2(t)}{(\gamma_2+f_2(t))(\gamma_2+g_2(t))} = s$. We show the complete construction of the distributed multiset equality check in Protocol 3.

**Protocol 3: Distributed multiset Equality Check PIOP**

$\mathcal{P}_0$ claims to $((\![f_1]\!], [\![f_2]\!], [\![g_1]\!], [\![g_2]\!], \mu, \nu); (f_1, f_2, g_1, g_2)) \in \mathcal{R}_{Muleq}$ to $\mathcal{V}$. $\mathcal{P}_i$ holds $f_1^{(i)}(x) = f_1(x, bin(i))$, $g_1^{(i)}(x) = g_1(x, bin(i))$, $f_2^{(i)}(x) = f_2(x, bin(i))$, $g_2^{(i)}(x) = g_2(x, bin(i))$.

1. $\mathcal{V}$ samples $\gamma_2 \leftarrow \mathbb{F}$ and sends $\gamma_2$ to $\mathcal{P}_0$. $\mathcal{P}_0$ transmits $\gamma_2$ to $\mathcal{P}_i$.
2. $\mathcal{P}_i$ computes $s^{(i)} \leftarrow \sum_{x \in \{0,1\}^{n-\log M}} (\frac{1}{\gamma_2 + f_1^{(i)}(x)} - \frac{1}{\gamma_2 + g_1^{(i)}(x)})$ and sends $s^{(i)}$ to $\mathcal{P}_0$. $\mathcal{P}_0$ computes $s \leftarrow \sum_{i \in [M]} s^{(i)}$.
3. $\mathcal{P}_0, \cdots, \mathcal{P}_{M-1}$ and $\mathcal{V}$ run the distributed rational sumcheck to check $((s, [\![g_1 - f_1]\!], [\![(\gamma_2 + f_1)(\gamma_2 + g_1)]\!]); (f_1, g_1)) \in \mathcal{R}_{RSUM}$ and $((s, [\![f_2 - g_2]\!], [\![(\gamma_2 + f_2)(\gamma_2 + g_2)]\!]); (f_2, g_2)) \in \mathcal{R}_{RSUM}$.

### 3.3 Distributed Spark Compiler

The Spark compiler[23] achieves linear prover time by utilizing a novel polynomial commitment scheme for a large but sparse matrix $M_P$, which efficiently verifies correct memory access patterns via an offline memory checking technique. $M_P$ is represented as 3 vectors $row, col, val$, which store the row indices, column indices, and values of its non-zero entries, respectively. To prove $\tilde{M_P}(\mathbf{u}_1, \mathbf{u}_2) = q'$, where $\mathbf{u}_1, \mathbf{u}_2 \in \mathbb{F}^n$, the prover asserts that:

$$(1) \sum_{\mathbf{x} \in \{0,1\}^{\log m}} \widetilde{erow}(\mathbf{x}) \cdot \widetilde{ecol}(\mathbf{x}) \cdot \widetilde{val}(\mathbf{x}) = q';$$

$$(2) \widetilde{erow}(\mathbf{x}) = \mathsf{eq}(\mathbf{u}_1, \widetilde{row}(\mathbf{x})), \widetilde{ecol}(\mathbf{x}) = \mathsf{eq}(\mathbf{u}_2, \widetilde{col}(\mathbf{x})).$$

We now describe the design of the distributed version of Spark. Relation (1) is handled by applying the standard distributed sumcheck. For the relation (2), we reduced it to the multiset equality check(Definition 9), as outlined below.

To prove $\widetilde{erow}(\mathbf{x}) = \mathsf{eq}(\mathbf{u}_1, \widetilde{row}(\mathbf{x}))$, the prover simulates a memory access procedure on an array of $n$ cells, indexed from 0 to $n-1$ in the head and the $i$-th cell stores $\mathsf{eq}(\mathbf{u}_1, \widetilde{row}(bin(i)))$. A global timestamp, initialized to 0, is introduced and incremented by one after each cell read. Whenever a cell is read, its timestamp is updated to this new value. The prover builds two vectors: $read\text{-}ts_{row} \in \mathbb{F}^m$ and $audit\text{-}ts_{row} \in \mathbb{F}^n$, where $k$-th entry of $read\text{-}ts_{row}$ is the read timestamp of $\mathsf{eq}(\mathbf{u}_1, \widetilde{row}(\mathbf{x}))$ at the time of the $k$-th read, and $audit\text{-}ts_{row}$ stores the final timestamp of each cell after all read operations are completed. It is sufficient to prove that $Init \cup WS = RS \cup Audit$, where:

$$Init = \{(i, \mathsf{eq}(\mathbf{u}_1, bin(i)), 0)\}_{i \in [n]}; RS = \{(\widetilde{row}(\mathbf{x}), \widetilde{erow}(\mathbf{x}), \widetilde{read\text{-}ts}(\mathbf{x}))\}_{\mathbf{x} \in \{0,1\}^{\log m}};$$

$$WS = \{(\widetilde{row}(\mathbf{x}), \widetilde{erow}(\mathbf{x}), \widetilde{read\text{-}ts}(\mathbf{x}) + 1)\}_{\mathbf{x} \in \{0,1\}^{\log m}}$$

$$Audit = \{(i, \mathsf{eq}(\mathbf{u}_1, bin(i)), \widetilde{audit\text{-}ts}(bin(i))\}_{i \in [n]};$$

We represent each tuple $(a, b, c)$ by evaluating the multiset hash $a \cdot \gamma_1^2 + b \cdot \gamma_1 + c$ under a random challenge $\gamma_1 \in \mathbb{F}$. This representation enables us to express the

relation $Init \cup WS = RS \cup Audit$ as a multiset equality check over these evaluated values, thereby reducing a complex set relation to a simpler algebraic statement. The detailed procedure is given in Protocol 4.

**Protocol 4: Distributed Spark**

$\mathcal{P}_0$ claims to $\mathcal{V}$ that $\tilde{M}_P(\mathbf{u}_1, \mathbf{u}_2) = q'$, where $M_P \in \mathbb{F}^{n \times n}$ is a sparse matrix with $m$ non-zero entries, and $\mathbf{u}_1, \mathbf{u}_2 \in \mathbb{F}^{\log n}$. Let $row, col, val$ denote a dense representation of $M$.

1. Running the memory-in-the-head procedure in Spark. Then $\mathcal{P}_i$ holds $\tilde{P}^{(i)}(\hat{\mathbf{u}}) = \tilde{P}(\hat{\mathbf{u}}, bin(i))$, where $\hat{\mathbf{u}} \in \mathbb{F}^{n - \log M}$, and $P \in \{row, col, val, read\text{-}ts_{row}, write\text{-}ts_{row}, audit\text{-}ts_{row}, read\text{-}ts_{col}, write\text{-}ts_{row}, audit\text{-}ts_{col}\}$.
2. $\mathcal{P}_0$ sends the oracle of $\widetilde{row}$,$\widetilde{col}$,$\widetilde{val}$,$\widetilde{read\text{-}ts_{row}}$,$\widetilde{write\text{-}ts_{row}}$,$\widetilde{audit\text{-}ts_{row}}$, $\widetilde{read\text{-}ts_{col}}$,$\widetilde{write\text{-}ts_{col}}$,$\widetilde{audit\text{-}ts_{col}}$ to $\mathcal{V}$.
3. $\mathcal{P}_0$ sends the oracle of $\widetilde{erow}, \widetilde{ecol}$ to $\mathcal{V}$, where

$$erow = [eq(row(0), \mathbf{u}_1), \ldots, eq(row(n-1), \mathbf{u}_1)]$$

$$ecol = [eq(col(0), \mathbf{u}_2), \ldots, eq(col(n-1), \mathbf{u}_2)]$$

4. $\mathcal{P}_0, \ldots, \mathcal{P}_{M-1}$ and $\mathcal{V}$ perform the distributed sumcheck for $q' = \sum_{k \in \{0,1\}^{\log n}} \widetilde{var}(\mathbf{u}) \cdot \widetilde{erow}(\mathbf{u}) \cdot \widetilde{ecol}(\mathbf{u})$.
5. $\mathcal{V}$ samples $\gamma_1 \in \mathbb{F}$ randomly and sends $\gamma_1$ to $\mathcal{P}_0$, and then $\mathcal{P}_0$ transmits $\gamma_1$ to $\mathcal{P}_i$.
6. $\mathcal{P}_0, \ldots, \mathcal{P}_{M-1}$ and $\mathcal{V}$ perform the distributed memory checking for $erow$ and $ecol$, which can be reduced to two checks for the multiset equality check relations
   $((([[Init_{row}]], [[WS_{row}]], [[Audit_{row}]], [[RS_{row}]], n, m);$
   $(Init_{row}, WS_{row}, Audit_{row}, RS_{row})) \in \mathcal{R}_{Muleq}$
   $((([[Init_{col}]], [[WS_{col}]], [[Audit_{col}]], [[RS_{col}]], n, m);$
   $(Init_{col}, WS_{col}, Audit_{col}, RS_{col})) \in \mathcal{R}_{Muleq}$, where:
   $Init_{row}(\mathbf{u}) = bin^{-1}(\mathbf{u}) \cdot \gamma_1^2 + \widetilde{mem}_{row}(\mathbf{u}) \cdot \gamma_1.$
   $RS_{row}(\mathbf{u}) = \widetilde{row}(\mathbf{u}) \cdot \gamma_1^2 + \widetilde{erow}(\mathbf{u}) \cdot \gamma_1 + \widetilde{read\text{-}ts_{row}}(\mathbf{u}).$
   $WS_{row}(\mathbf{u}) = \widetilde{row}(\mathbf{u}) \cdot \gamma_1^2 + \widetilde{erow}(\mathbf{u}) \cdot \gamma_1 + \widetilde{write\text{-}ts_{row}}(\mathbf{u}).$
   $Audit_{row}(\mathbf{u}) = bin^{-1}(\mathbf{u}) \cdot \gamma_1^2 + \widetilde{mem}_{row}(\mathbf{u}) \cdot \gamma_1 + \widetilde{audit\text{-}ts_{row}}(\mathbf{u}).$
   $Init_{col}(\mathbf{u}) = bin^{-1}(\mathbf{u}) \cdot \gamma_1^2 + \widetilde{mem}_{col}(\mathbf{u}) \cdot \gamma_1.$
   $RS_{col}(\mathbf{u}) = \widetilde{col}(\mathbf{u}) \cdot \gamma_1^2 + \widetilde{ecol}(\mathbf{u}) \cdot \gamma_1 + \widetilde{read\text{-}ts_{col}}(\mathbf{u}).$
   $WS_{col}(\mathbf{u}) = \widetilde{col}(\mathbf{u}) \cdot \gamma_1^2 + \widetilde{ecol}(\mathbf{u}) \cdot \gamma_1 + \widetilde{write\text{-}ts_{col}}(\mathbf{u}).$
   $Audit_{col}(\mathbf{u}) = bin^{-1}(\mathbf{u}) \cdot \gamma_1^2 + \widetilde{mem}_{col}(\mathbf{u}) \cdot \gamma_1 + \widetilde{audit\text{-}ts_{col}}(\mathbf{u}).$
   $\widetilde{mem}_{row}(\mathbf{u}) = \mathsf{eq}(\mathbf{u}_1, \mathbf{u})$; $\widetilde{mem}_{col}(\mathbf{u}) = \mathsf{eq}(\mathbf{u}_2, \mathbf{u})$.

### 3.4 Distributed Spartan PIOP

In Spartan PIOP, the provers and verifiers execute two sumcheck protocols in sequence:

1. $\sum_{x\in\{0,1\}^{\log n}}(\overline{A}(x)\cdot\overline{B}(x)-\overline{C}(x))\cdot e_\tau(x)=0$ for $P=\{A,B,C\}$, where $\overline{P}(x)=\sum_{y\in\{0,1\}^{\log m}}\tilde{P}(x,y)\tilde{z}(y)$, for $P=\{A,B,C\}$, and $e_\tau(x)=\mathsf{eq}(x,\tau)$.
2. $\sum_{y\in\{0,1\}^{\log m}}\tilde{P}(r_{\mathbf{x}},y)\tilde{z}^{(i)}(y)=v_P$, for $P\in\{A,B,C\}$.

The three instances in sumcheck 2 can be batched into a single sumcheck protocol via a random linear combination, using the randomness $r_A, r_B, r_C$ generated by the verifier.

In a distributed setting with $M$ sub-provers, we employ the distributed sumcheck protocol from [15] (step 3 and step 9) to handle the sumcheck protocol 1 and 2. This requires each sub-prover $\mathcal{P}_i$ to compute the partial polynomials $\overline{P}^{(i)}(x)=\overline{P}(x,bin(i))$ for sumcheck protocol 1 and the partial polynomials $\tilde{P}^{(i)}_{\mathbf{r}_x,Y}(y)=\tilde{P}(\mathbf{r_x},y||bin(i))$ for sumcheck protocol 2, for $P\in\{A,B,C\}$. This approach is made feasible by a strategic distribution of the matrices $A$, $B$, $C$, and the vector $z$ among the sub-provers, enabling each $\mathcal{P}_i$ to efficiently compute its required partial polynomial for the associated sumcheck protocol(step 1 and step 8). We detail this process in the following.

To minimize memory usage per prover, the matrices $A, B$ and $C$, along with the vector $z$, are distributed among the sub-provers $\mathcal{P}_i$ before protocol execution. Specifically, each matrix $P\in\{A,B,C\}$ is partitioned row-wise into $\{P_1^{(i)}\}_{i\in[M]}$ and column wise into $\{P_2^{(i)}\}_{i\in[M]}$. For $P\in\{A,B,C\}$. The sub-matrices $P_1^{(i)}$ and $P_2^{(i)}$ are then assigned to sub-prover $\mathcal{P}_i$, where $P_1^{(i)}, P_2^{(i)}$ are utilized by $\mathcal{P}_i$ to compute the partial polynomials for sumcheck 1 and sumcheck 2, respectively. For the vector $z$, in order to compute the partial polynomials $\overline{A}_1^{(i)},\overline{B}_1^{(i)},\overline{C}_1^{(i)}$ required for the distributed sumcheck protocol 1, each prover $\mathcal{P}_i$ only needs entries of $z$ corresponding to the non-zero positions in $A_1^{(i)}, B_1^{(i)}, C_1^{(i)}$. We denote the set of indices needed for $A_1^{(i)}$ as $S_A^{(i)}$, and similarly define $S_B^{(i)}, S_C^{(i)}$ for $B^{(i)}$ and $C^{(i)}$, respectively. Additionally, $\mathcal{P}_i$ stores a segment $z^{(i)}=z[i\overline{n}:(i+1)\overline{n}-1]$ of the vector $z$ for use in computing the partial polynomials required by the distributed sumcheck protocol 2.

$\mathcal{P}_i$ computes the partial polynomial $\tilde{P}^{(i)}(r_{\mathbf{x}},y)=\sum_{\mathbf{b}\in\{0,1\}^{\log n}}\tilde{P}_2^{(i)}(\mathbf{b},y)\mathsf{eq}(\mathbf{b},\mathbf{r}_x)$ over $y\in\{0,1\}^{\log n}$ as required by the distributed sumcheck protocol 2, where $P\in\{A,B,C\}$. This computation is performed efficiently by leveraging the sparsity of $\tilde{A}_2^{(i)},\tilde{B}_2^{(i)},\tilde{C}_2^{(i)}$, since only their non-zero entries need to be processed. To compute $\mathsf{eq}(\mathbf{b},\mathbf{r}_x)$, the sub-prover generates $\mathbf{e}_1=\otimes_{i=1}^{\log\overline{n}}(1-r_i,r_i)$, while the master prover $\mathcal{P}_0$ generates the vector $\mathbf{e}_2=\otimes_{i=\log\overline{n}+1}^{\log n}(1-r_i,r_i)$ and transmits $\mathbf{e}_2$ to all sub-provers. Let $\mathbf{e}_{r_x}\in\mathbb{F}^n$ denote the vector storing all evaluations of $\mathsf{eq}(\mathbf{b},\mathbf{r}_x)$, then $\mathbf{e}_{r_x}=\mathbf{e}_1\otimes\mathbf{e}_2$. Accordingly, each sub-prover $\mathcal{P}_i$ can compute $\mathsf{eq}(\mathbf{b},\mathbf{r}_x)$ for any $\mathbf{b}$ by looking up the corresponding entries in of $\mathbf{e}_1$ and $\mathbf{e}_2$ and multiplying them together.

Following the sumcheck phase, the verifier's queries to the multilinear polynomials $\tilde{A}, \tilde{B}, \tilde{C}$ are handled using our distributed Spark (Protocol 4), which provides an efficient commitment to the sparse polynomial $\tilde{A}, \tilde{B}$ and $\tilde{C}$(step 11).

The full construction of the distributed Spartan PIOP is presented in Protocol 5.

**Protocol 5: Distributed Spartan PIOP**

$\mathcal{P}_0$ claims to $\mathcal{V}$ that $Az \circ Bz = Cz$, where $A, B, C \in \mathbb{F}^{n \times n}, z = (1, io, w)$ and $z \in \mathbb{F}^n$. Let $\overline{n} := \frac{n}{M}$. $\mathcal{P}_i$ holds:

- $\{z[p] | p \in S_A^{(i)} \cup S_B^{(i)} \cup S_C^{(i)} \cup [i\overline{n} : (i+1)\overline{n} - 1]\}$, where $S_P^{(i)} = \{k | \exists t \in [\overline{n}], P^{(i)}[t, j] \neq 0\}$ for $P \in \{A, B, C\}$.
- $z^{(i)} \in \mathbb{F}^{\overline{n}}$, where $z^{(i)} = z[i\overline{n} : (i+1)\overline{n} - 1]$.
- $A_1^{(i)}, B_1^{(i)}, C_1^{(i)} \in \mathbb{F}^{\overline{n} \times n}$, where $P_1^{(i)} = P[j :]$, for $P \in \{A, B, C\}, j \in [i\overline{n} : (i+1)\overline{n} - 1]$.
- $A_2^{(i)}, B_2^{(i)}, C_2^{(i)} \in \mathbb{F}^{n \times \overline{n}}$, where $P_2^{(i)} = P[: j]$, for $P \in \{A, B, C\}, j \in [i\overline{n} : (i+1)\overline{n} - 1]$.

1. $\mathcal{P}_i$ computes $\overline{P}^{(i)}(x) \leftarrow \sum_{y \in \{0,1\}^{\log n}} P_1^{(i)}(x, y)\tilde{z}^{(i)}(y)$, for $P \in \{A, B, C\}$.
2. $\mathcal{V}$ samples $\tau \in \mathbb{F}$ and sends $\tau$ to $\mathcal{P}_0$, and $\mathcal{P}_0$ transmits $\tau$ to $\mathcal{P}_i$.
3. $\mathcal{P}_0, \ldots, \mathcal{P}_{M-1}$ and $\mathcal{V}$ run the first sumcheck for $\sum_{x \in \{0,1\}^{\log n}} (\overline{A}(x) \cdot \overline{B}(x) - \overline{C}(x)) \cdot e_\tau(x) = 0$, where $\overline{P}(x) = \sum_{y \in \{0,1\}^{\log n}} \tilde{P}_1(x, y)\tilde{z}(y)$, for $P = \{A, B, C\}$, and $e_\tau(x) = \mathsf{eq}(x, \tau)$. $\mathcal{P}_i$ holds $e_\tau^{(i)}(x) := e_\tau(x, bin(i))$ and $\overline{P}^{(i)}(x)$, for $P \in \{A, B, C\}$. At last $\mathcal{P}_0$ claims to $\mathcal{V}$ that $Q_1(\mathbf{r}_x) = e_x$, where $Q_1(x) = (\overline{A}(x) \cdot \overline{B}(x) - \overline{C}(x)) \cdot e_\tau(x)$.
4. $\mathcal{P}_0$ computes $v_A \leftarrow \sum_{i \in [M]} \tilde{A}_1^{(i)}(\mathbf{r}_x), v_B \leftarrow \sum_{i \in [M]} \tilde{B}_1^{(i)}(\mathbf{r}_x), v_C \leftarrow \sum_{i \in [M]} \tilde{C}_1^{(i)}(\mathbf{r}_x)$, where $\mathbf{r}_x := (r_1, \ldots, r_{\log n})$. Then $\mathcal{P}_0$ sends $(v_A, v_B, v_C)$ to $\mathcal{V}$.
5. $\mathcal{V}$ checks $(v_A \cdot v_B - v_C) \cdot e_\tau(\mathbf{r}_x) = e_x$, abort if the check fails.
6. $\mathcal{V}$ samples $r_A, r_B, r_C \leftarrow \mathbb{F}$ to $\mathcal{P}_0$, and $\mathcal{P}_0$ sends $\overline{v}$ to $\mathcal{V}$, where $\overline{v} = r_A \cdot v_A + r_B \cdot v_B + r_C \cdot v_C$.
7. Each sub-prover $\mathcal{P}_i$ generates the vector $\mathbf{e}_1 = \otimes_{i=1}^{\log \overline{n}}(1 - r_i, r_i)$, while the master prover $\mathcal{P}_0$ generates the vector $\mathbf{e}_2 = \otimes_{i=\log \overline{n}+1}^{\log n}(1 - r_i, r_i)$ and transmits $\mathbf{e}_2$ to $\mathcal{P}_i$.
8. (a) For a given binary vector $\mathbf{b}_x \in \{0,1\}^{\log n}$, each $\mathcal{P}_i$ looks up the vector $\mathbf{e}_1$ and $\mathbf{e}_2$ to obtain $b_1 = \mathbf{e}_1[\mathbf{b}_x[0 : \log \overline{n}]]$ and $b_2 = \mathbf{e}_2[\mathbf{b}_x[\log \overline{n} + 1 : \log n]]$, then computes $\mathsf{eq}(\mathbf{b}_x, \mathbf{r}_x) = b_1 b_2$.
   (b) For $P \in \{A, B, C\}$, $\mathcal{P}_i$ computes the partial polynomial $\tilde{P}^{(i)}(r_x, y) = \sum_{\mathbf{b} \in \{0,1\}^{\log n}} \tilde{P}_2^{(i)}(\mathbf{b}, y)\mathsf{eq}(\mathbf{b}, \mathbf{r}_x)$ for $\tilde{P}(\mathbf{r}_x, y)$ by summing the product of the non-zero entry $\tilde{P}_2^{(i)}(\mathbf{b}', y)$ and $\mathsf{eq}(\mathbf{b}', \mathbf{r}_x)$ over all $y \in \{0,1\}^{\log \overline{n}}$, where $\mathsf{eq}(\mathbf{b}', \mathbf{r}_x)$ is computed using the method in 8a .

9. Let $\tilde{P}_{\mathbf{r}_x,Y}(y) := \tilde{P}(\mathbf{r}_x, y)\tilde{z}(y)$ for $P \in \{A, B, C\}$, such $\mathcal{P}_i$ holds $\tilde{P}^{(i)}_{\mathbf{r}_x,Y}(y) = \tilde{P}_{\mathbf{r}_x,Y}(y, bin(i))$, for $P \in \{A, B, C\}$. $\mathcal{P}_i$ and $\mathcal{V}$ run the second sumcheck for $\sum_{y\in\{0,1\}^{\log n}} Q_2(y) = \overline{v}$, where $Q_2(y) = r_A \cdot \tilde{A}_{\mathbf{r}_x,Y}(y) + r_B \cdot \tilde{B}_{\mathbf{r}_x,Y}(y) + r_C \cdot \tilde{C}_{\mathbf{r}_x,Y}(y)$. At last $\mathcal{P}_0$ claims to $\mathcal{V}$ that $Q_2(\mathbf{r}_y) = e_y$.
10. $\mathcal{P}_0, \ldots, \mathcal{P}_{M-1}$ and $\mathcal{V}$ runs the distributed Spark(Protocol 4) to reduce the proof of $\tilde{A}(\mathbf{r}_x, \mathbf{r}_y) = v_{A,y}, \tilde{B}(\mathbf{r}_x, \mathbf{r}_y) = v_{B,y}, \tilde{C}(\mathbf{r}_x, \mathbf{r}_y) = v_{C,y}$ to the queries of several polynomials with $\log n$ variates. $\mathcal{P}_0$ sends $v_{A,y}, v_{B,y}, v_{C,y}$ to $\mathcal{V}$, and $\mathcal{V}$ checks $r_A v_{A,y} v_Z + r_B v_{B,y} v_Z + r_C v_{C,y} v_Z = \overline{v}$. If the check fails, abort.
11. $\mathcal{V}$ query $\tilde{w}$ at $\mathbf{r}_y[1..]$, and compute $v_Z \leftarrow (1 - \mathbf{r}_y[0]) \cdot \tilde{w}(\mathbf{r}_y[1..]) + \mathbf{r}_y[0] \cdot \widetilde{(io, 1)}(\mathbf{r}_y[1..])$.

### 3.5 Security and Complexity Analysis

**Security** We conduct our security analysis under the same trust assumptions as the single-prover setting, given that the sub-provers in our system are trusted entities and the distribution is intended solely for acceleration. We begin by analyzing the security of the distributed multiset equality check (Protocol 3), followed by the distributed Spark (Protocol 4), and finally the distributed Spartan (Protocol 5).

*Security of distributed multiset equality check* Completeness follows straightforwardly from the protocol design. For knowledge soundness, Lemma 1 establishes the equivalence between the multiset equality check and Equation 3, which in turn is equivalent to Equation 4. By the Schwartz-Zippel lemma, if Equation 5 holds, then Equation 4 holds with probability $1 - \mathsf{negl}(\lambda)$. Furthermore, Equation 5 is equivalent to Equation 6, which reduces to two invocations of the rational sumcheck relation. Consequently, the soundness of the distributed rational sumcheck protocol guarantees the soundness of the multiset equality check.

*Security of distributed Spark* The completeness is from the completeness of memory checking and the sumcheck protocol. For the knowledge soundness, we first quote Lemma 7.3 in [23], whose proofs can be found accordingly and are hence omitted for brevity.

**Lemma 2 ([23]).** *Let $h_\gamma : \mathbb{F}^3 \to \mathbb{F}$ be a hash function, where $h_\gamma(a, v, t) = a\gamma^2 + v\gamma + t$. Let $H_\gamma : (\mathbb{F}^\ell, \mathbb{F}^\ell, \mathbb{F}^\ell) \to \mathbb{F}^\ell$ be a map, where $H_\gamma(A, V, T) = [h_\gamma(A[0], V[0], T[0]), \ldots, h_\gamma(A[\ell-1], V[\ell-1], T[\ell-1])]$. For any $\ell > 0$, $(A_1, V_1, T_1) \in (\mathbb{F}^\ell, \mathbb{F}^\ell, \mathbb{F}^\ell)$ and $(A_2, V_2, T_2) \in (\mathbb{F}^\ell, \mathbb{F}^\ell, \mathbb{F}^\ell)$,*

$$\Pr_\gamma\{\exists i : H_\gamma(A_1, V_1, T_1)[i] = H_\gamma(A_2, V_2, T_2)[i] \mid (A_1, V_1, T_1) \neq (A_2, V_2, T_2)\} \leq 3 \cdot \ell / |\mathbb{F}|$$

By invoking Lemma 2 and the soundness of memory checking [3], the verification of $\tilde{P}(\mathbf{r}_x, \mathbf{r}_y) = v_{P,y}$ for $P \in \{A, B, C\}$ reduces to a multiset equality check. Consequently, the knowledge soundness of the distributed Spark protocol follows directly from the knowledge soundness of the multiset equality check established above.

*Security of distributed Spartan PIOP* Our protocol differs from the original Spartan protocol [23] primarily in the construction of the Spark component, which we implement via a multiset equality check (Definition 9). To illustrate the security reduction, consider a modification of Protocol 5 where Step 10 is replaced with the following interaction:
$\mathcal{P}_i$ *sends* $\tilde{A}, \tilde{B}, \tilde{C}$ *to* $\mathcal{V}$*, and* $\mathcal{V}$ *computes* $v_{A,y} \leftarrow \tilde{A}(\boldsymbol{r}_x, \boldsymbol{r}_y)$, $v_{B,y} \leftarrow \tilde{B}(\boldsymbol{r}_x, \boldsymbol{r}_y)$, $v_{C,y} \leftarrow \tilde{C}(\boldsymbol{r}_x, \boldsymbol{r}_y)$.
This modified protocol is identical to the one described in Section 5.1 of [23], with its security analysis provided in Appendix A of [23]. Therefore, assuming an extractable polynomial commitment scheme[3], it suffices to establish the completeness and knowledge soundness of the Spark protocol. Consequently, by combining the baseline security of Spartan with the security of the distributed Spark protocol established above, we obtain the security of the distributed Spartan PIOP.

**Complexity** The complexity analysis of our distributed Spartan PIOP(Protocol 5) is summarized as follows.

*Prover time* Each $\mathcal{P}_i$ performs $O(\frac{n}{M})$ field operations on average to compute $\tilde{P}_1^{(i)}(x)$ for $P \in \{A, B, C\}$ in step 1, since the sub-matrix $P_1^{(i)}(x)$ contains $O(\frac{n}{M})$ non-zero entries. $\mathcal{P}_i$ and $\mathcal{V}$ execute two distributed sumcheck protocols in step 3 and step 9, where $\mathcal{P}_i$ also performs $O(\frac{n}{M})$ field operations. In step 8, each $\mathcal{P}_i$ performs $O(\frac{n}{M})$ field operations to generates the vector $\mathbf{s}_1$ and another $O(\frac{n}{M})$ field operations to computes $\tilde{P}^{(i)}_{\mathbf{r}_x,Y}$ using $\mathbf{s}_1$ and $\mathbf{s}_2$.

*Verifier time* $\mathcal{V}$ performs $O(\log m) + O(\log n)$ field operations across the two sumcheck protocols in step 3 and step 9.

*Communication cost* In step 8, $\mathcal{P}_0$ sends the vector $\mathbf{s}_2$ to every $\mathcal{P}_i$, incurring a communication cost of $O(M^2)$. The two sumcheck protocols in step 3 and step 9 involves $O(\log m) + O(\log n)$ communication between $\mathcal{P}_0$ and $\mathcal{V}$.

*Oracle and Query* In step 10, $\mathcal{P}_i$ sends the oracles of $\tilde{A}$, $\tilde{B}$ and $\tilde{C}$ to $\mathcal{V}$, which corresponds to $O(1)$ queries over $\log m$ or $\log n$ variates, as specified in the distributed Spark protocol (Protocol 4). In step 11, $\mathcal{P}_0$ sends the oracle for $\tilde{w}$ to $\mathcal{V}$. The verifier $\mathcal{V}$ makes $O(1)$ queries to these oracles.

[3] In [23], an extractable polynomial commitment scheme is defined as having completeness, binding, and knowledge soundness properties. Our work adopts this same definition.

Overall, each $\mathcal{P}_i$ performs $O(\frac{n}{M})$ field operations and $\mathcal{V}$ performs $O(\log m) + O(\log n)$ field operations. The total communication size is $O(M^2) + O(\log m) + O(\log n)$. Additionally, the $\mathcal{P}_0$ sends $O(1)$ oracle to $\mathcal{V}$ and $\mathcal{V}$ makes $O(1)$ queries to these oracles.

# 4 Distributed Transparent PCS with Linear Prover Time

## 4.1 Brakedown with Proof Composition

Our starting point is Brakedown[12] polynomial commitment scheme, which employs a linear time encoding scheme $Enc : \mathbb{F}^n \to \mathbb{F}^{cn}$, where $c$ is the reciprocal of the rate of the code. Below, we summarize the Brakedown procedure for proving that $f(\mathbf{s}) = v$, where $f$ is a $\mu$-variate multi-linear polynomial. Let $\mathbf{s} = \mathbf{s}_1 || \mathbf{s}_2$, where $\mathbf{s}_1 \in \mathbb{F}^{\log n_1}$ and $\mathbf{s}_2 \in \mathbb{F}^{\log n_2}$.

In the commitment phase, the evaluations of $f$ are arranged into a matrix $M_f \in \mathbb{F}^{n_1 \times n_2}$, where $n_1 n_2 = 2^\mu = N$. The prover computes an encoded matrix $M'_f \in \mathbb{F}^{n_1 \times cn_2}$, where each row $M'_f[i:]$ equals $Enc(M_f[i:])$ for all $i \in [n_1]$. Then the prover computes the Merkle commitment of $M'_f$ as the commitment.

The evaluation phase consists of two main steps. The first step is proximity test: the verifier sends a randomly sampled vector $\mathbf{a} \in \mathbb{F}^{n_1}$ to the prover, which responds with the linearly combined vectors $F_1, F_2 \in \mathbb{F}^{n_2}$ to the verifier, where $F_1[j] = \sum_{i \in [n_1]} \mathbf{a}[i] M_f[i,j]$ and $F_2[j] = \sum_{i \in [n_1]} \mathsf{eq}(\mathbf{s}_1, bin(i)) M_f[i,j]$. The verifier then samples a random index set $\mathcal{I} \subseteq [n_2]$ of size $\ell = \Theta(\lambda)$ and sends it to the prover. The prover returns the columns of $M'_f$ for the indices in $\mathcal{I}$, denoted as $R \in \mathbb{F}^{n_1 \times \ell}$, along with the corresponding Merkle proofs. The verifier checks that: (1) the validity of all Merkle proofs, and (2) $E_1[i] = \sum_{j \in [n_2]} \mathbf{a}[j] R[j,i]$ for each $i \in [\ell]$, where $E_1 = Enc(F_1)$. This process is repeated with a fresh $\mathcal{I}$, replacing $F_1$ with $F_2$. The second step is the evaluation check, in which the verifier checks that $v = F_2[i] \cdot \mathsf{eq}(\mathbf{s}_2, bin(i))$. The proof size is given by $max\{O(n_2), O(\lambda n_1)\}$. By optimizing the choice of $n_1$, $n_2$, the proof size is minimized to $O(\sqrt{\lambda N})$. The verifier's runtime is dominated by computing $E_1$, leading to a complexity of $O(\sqrt{N})$.

We show the Brakedown polynomial commitment with proof composition, as primarily outlined in [20] and [21]. The coefficients of the polynomial $f$ are arranged into a matrix of dimensions $B \times \frac{N}{B}$ (typically $B \in [\log N : \sqrt{N}]$), such that $\mathbf{s}_1 \in \mathbb{F}^{\log B}$ and $\mathbf{s}_2 \in \mathbb{F}^{\log \frac{N}{B}}$.

There are two differences from the original Brakedown[12] construction: (1) Instead of sending the full vectors $E_1, F_1, E_2, F_2 \in \mathbb{F}^{\frac{N}{B}}$ directly after linear combination, $\mathcal{P}$ sends to $\mathcal{V}$ the commitment to the $\tilde{E}_1, \tilde{F}_1, \tilde{E}_2, \tilde{F}_2$ and claims that $E_1 = Enc(F_1), E_2 = Enc(F_2)$(as formally defined as $\mathcal{R}$) in step 9, and $\mathcal{V}$ verifies the claim is valid. (2)To check $v = \sum_{i \in [\frac{N}{B}]} F_2[i] \cdot \mathsf{eq}(\mathbf{s}_2, bin(i))$, the prover opens the commitment to $\tilde{F}_2$ at the point $\mathbf{s}_2$ and the verifier check the opening proof. The full construction is detailed in Protocol 6.

**Protocol 6: Brakedown with Proof Composition**

$\mathcal{P}$ claims $f(\mathbf{s}) = v$ to $\mathcal{V}$, where $f \in \mathcal{F}^{(\leq 1)}_{\log N}$, and $\mathbf{s} = \mathbf{s}_1 || \mathbf{s}_2$, $\mathbf{s}_1 \in \mathbb{F}^{\log B}$, $\mathbf{s}_2 \in \mathbb{F}^{\log \frac{N}{B}}$. $PC = (Gen, Commit, Open, Eval)$ is the underlying polynomial commitment. $MT = (Commit, Eval)$ is Merkle commitment. $Enc : \mathbb{F}^{\frac{N}{B}} \to \mathbb{F}^{\frac{cN}{B}}$ is the map of the encoding algorithm for the linear code in Brakedown [12]. Let $H(\cdot) : \mathbb{F}^* \to \mathbb{F}$ be a random oracle.

Commit phase:

**Input**: the public parameters $\mathsf{pp}$, the multilinear polynomial $f$.

**Output**: the commitment $C$.

Arrange the matrix of $f$ in a matrix $M_f \in \mathbb{F}^{B \times \frac{N}{B}}$. $\mathcal{P}$ computes $M'_f \in \mathbb{F}^{B \times \frac{cN}{B}}$ such that $M'_f[k :] \leftarrow Enc(M_f[k :])$. Let $\mathbf{h} \in \mathbb{F}^{cN/B}$ be a vector such that $\mathbf{h}[i] \leftarrow H(M'_f[: i])$, for each $i \in [cN/B]$. $\mathcal{P}$ computes $C \leftarrow MT.Commit(\mathbf{h})$.

Eval phase:

**Public input:** the public parameters $\mathsf{pp}$, the commitment $C$, evaluation point $\mathbf{s} \in \mathbb{F}^{\log N}$, $v$.

**Private input:** the multilinear polynomial $f$.

1. $\mathcal{V} \to \mathcal{P}$ : a random vector $\mathbf{a} \in \mathbb{F}^B$.
2. $\mathcal{P} \to \mathcal{V}$ : the commitments $C_{E1}, C_{F1}, C_{E2}, C_{F2}$ to the multi-linear extension of $E_1, F_1, E_2, F_2$ using $PC.Commit$, where $E_1 = Enc(F_1)$, $F_1 = \sum_{i \in [B]} \mathbf{a}[i] M_f[i]$, $E_2 = Enc(F_2)$, $F_2 = \sum_{i \in [B]} \mathsf{eq}(\mathbf{s}_1, i) M_f[i]$.
3. $\mathcal{P} \to \mathcal{V}$ : the merkle-tree roots $rt_1 \leftarrow MT.Commit(E_1)$ and $rt_2 \leftarrow MT.Commit(E_2)$.
4. $\mathcal{V} \to \mathcal{P}$ : the column indexes $\mathcal{I} = \{idx_1, \ldots, idx_\ell\}$.
5. $\mathcal{P} \to \mathcal{V}$ : columns $R \in \mathbb{F}^{\ell \times B}$ such that $R[j :] = M'_f[: idx_j]$, for all $idx_j \in \mathcal{I}$, vectors $\overline{\mathbf{h}}$ and proofs $\{\Pi_j\}_{j \in [\ell]}$ for each selected column to $\mathcal{V}$, where $\Pi_j \leftarrow MT.Eval.Prove(\mathbf{h}, idx_j, \overline{\mathbf{h}}[j])$ and $\overline{\mathbf{h}}[j] = H(R[j :])$ for $j \in [\ell]$.
6. $\mathcal{P} \to \mathcal{V}$ : vectors $\mathbf{y}_1, \mathbf{y}_2$ such that $\mathbf{y}_1[i] = \sum_{j \in [B]} R[i, j] \cdot \mathbf{a}[j]$ , $\mathbf{y}_2[i] = \sum_{j \in [B]} R[i, j] \cdot \mathsf{eq}(\mathbf{s}_1, j)$ for $i \in [\ell]$.
7. $\mathcal{P} \to \mathcal{V}$ : the proof of polynomial commitment $\pi_{F2} \leftarrow PC.Eval.Prove(F_2, \mathbf{s}_2, v)$.
8. $\mathcal{P} \to \mathcal{V}$ : the proof of merkle-hashing commit $\{\Pi_{1,j}, \Pi_{2,j}\}_{j \in [\ell]}$, where $\Pi_{1,j} \leftarrow MT.Eval.Prove(E_1, idx_j, e_{1,j})$, $\Pi_{2,j} \leftarrow MT.Eval.Prove(E_2, idx_j, e_{2,j})$, for $j \in [\ell]$, and the claimed value $\{e_{1,j}, e_{2,j}\}_{j \in [\ell]}$.
9. $\mathcal{P} \to \mathcal{V}$ : the proof $\pi_r$ for the relation $\mathcal{R}$, where $\mathcal{R} = \{(C_{E1}, C_{F1}, C_{E2}, C_{F2}; E_1, F_1, E_2, F_2) | E_1 = Enc(F_1) \wedge E_2 = Enc(F_2) \wedge C_{E1} = PC.Commit(\tilde{E}_1) \wedge C_{F1} = PC.Commit(\tilde{F}_1) \wedge C_{E2} = PC.Commit(\tilde{E}_2) \wedge C_{F2} = PC.Commit(\tilde{F}_2)\}$.
10. $\mathcal{V}$ checks:
    (a) $MT.Eval.Verify(C, \pi_j, \overline{\mathbf{h}}[j], idx_j) = 1$, for $j \in [\ell]$.
    (b) $\mathbf{y}_1[i] = \sum_{j \in [B]} R[i, j] \cdot \mathbf{a}[j]$ , $\mathbf{y}_2[i] = \sum_{j \in [B]} R[i, j] \cdot \mathsf{eq}(\mathbf{s}_1, j)$ for $i \in [\ell]$.

(c) $PCS.Eval.Verify(C_{F2}, \pi_{F2}, v, \mathbf{s}_2) = 1$
(d) $\mathbf{y}_1[j] = e_{1,j}, \mathbf{y}_2[j] = e_{2,j}$, $MT.Verify(rt_1, \Pi_{1,j}, e_{1,j}, idx_j) = 1$, $MT.Eval.Verify(rt_2, \Pi_{2,j}, e_{2,j}, idx_j) = 1$, for $j \in [\ell]$.
(e) the relation $\mathcal{R}$ is valid with the proof $\pi_r$.

There are two main challenges when considering the Brakedown protocol in a distributed setting: (1) handling commitments to linearly combined vectors, and (2) proving the relation $\mathcal{R}$. These are discussed in Subsection 4.2 and Subsection 4.3, respectively.

### 4.2 Distributed Commitments to Combined Vectors in Brakedown

This subsection discusses the distributed generation of commitments, specifically the Merkle hash commitment and the polynomial commitment for the linearly combined vectors $E_1$, $F_1$, $E_2$, $F_2 \in \mathbb{F}^{\frac{N}{B}}$, where $E_1 = Enc(F_1)$, $F_1 = \sum_{i\in[B]} \mathbf{a}[i]M_f[i]$, $E_2 = Enc(F_2)$, $F_2 = \sum_{i\in[B]} \mathsf{eq}(\mathbf{s}_1, i)M_f[i]$.

**Merkle commitment** In the Merkle commitment scheme, the prover claims to the verifier that $E_1[idx] = e_1, E_2[idx] = e_2$, where $idx \in [\ell]$, and $E_1, E_2$ are linearly combined vectors in Protocol 6, which are committed using Merkle commitment. In the distributed setting, the matrix is partitioned row-wise and evenly distributed among the provers. Each sub-prover $\mathcal{P}_i$ computes the partial vectors $E_1^{(i)} = \sum_{k\in[b]} \mathbf{a}^{(i)}[k]Enc(M_f^{(i)})[k]$, and $E_2^{(i)} = \sum_{k\in[b]} \mathsf{eq}(\mathbf{s}_1, bin(ib + k)) \cdot Enc(M_f^{(i)})[k]$. Then $\mathcal{P}_i$ constructs a local Merkle root for $E_1^{(i)}, E_2^{(i)}$, denoted $rt_{E1}^{(i)}, rt_{E2}^{(i)}$, and sends them to the master prover $\mathcal{P}_0$. $\mathcal{P}_0$ forwards the set of all roots $\{rt_{E1}^{(i)}, rt_{E2}^{(i)}\}_{i\in[M]}$ to the verifier $\mathcal{V}$. During the opening phase, each sub-prover supplies a proof that its Merkle root correctly commits to the corresponding vector $E_1^{(i)}, E_2^{(i)}$. The verifier individually checks each of these proofs, and finally verifies that $e_1 = \sum_{i\in[M]} e_1^{(i)}$ and $e_2 = \sum_{i\in[M]} e_2^{(i)}$.

We show the full protocol in Protocol 7.

**Protocol 7: Distributed Merkle Commitments to $E_1$, $E_2$.**
$\mathcal{P}_0$ claims to $\mathcal{V}$ that $E_1[idx] = e_1, E_2[idx] = e_2$. $\mathcal{P}_i$ holds sub-matrix $M_f^{(i)} \in \mathbb{F}^{b\times\frac{N}{B}}$ and $\mathbf{a}^{(i)} \in \mathbb{F}^b$, where $M_f^{(i)}[k] = M_f[ib + k :], \mathbf{a}^{(i)}[k] = \mathbf{a}[ib + k]$, and $b = \frac{B}{M}$. Let $\beta^{(i)} \in \mathbb{F}^b$ where $\beta^{(i)}[k] = \mathsf{eq}(\mathbf{s}_1, bin(ib + k))$. $MT = (Commit, Eval)$ is Merkle commitment. $Enc : \mathbb{F}^{\frac{N}{B}} \to \mathbb{F}^{\frac{cN}{B}}$ is the map of the encoding algorithm for the linear code in Brakedown [12].
Commit phase:

1. $\mathcal{P}_i$ computes $M_f'^{(i)} \in \mathbb{F}^{b \times N/B}$ such that $M_f'^{(i)}[k:] = Enc(M_f^{(i)}[k:])$, for $k \in [b]$. Then $\mathcal{P}_i$ computes $E_1^{(i)}, E_2^{(i)}$, where $E_1^{(i)} = \sum_{k \in [b]} \mathbf{a}^{(i)}[k] M_f'^{(i)}[k:]$, and $E_2^{(i)} = \sum_{k \in [b]} \beta^{(i)}[k] M_f'^{(i)}[k:]$.
2. $\mathcal{P}_i$ sends the hash roots $rt_{E1}^{(i)} \leftarrow MT.Commit(E_1^{(i)})$ and $rt_{E2}^{(i)} \leftarrow MT.Commit(E_2^{(i)})$ to $\mathcal{P}_0$, and $\mathcal{P}_0$ outputs $\{rt_{E1}^{(i)}, rt_{E2}^{(i)}\}_{i \in [M]}$.

Prove phase:

$\mathcal{P}_i$ sends the Merkle proofs $\Pi_{E1}^{(i)} \leftarrow MT.Eval.Prove(E_1^{(i)}, idx, e_1^{(i)})$ and $\Pi_{E2}^{(i)} \leftarrow MT.Eval.Prove(E_2^{(i)}, idx, e_2^{(i)})$ and the corresponding claimed value $e_1^{(i)}$ and $e_2^{(i)}$ to $\mathcal{P}_0$, and $\mathcal{P}_0$ sends $\{\Pi_{E1}^{(i)}, \Pi_{E2}^{(i)}\}_{i \in [M]}$ and $\{e_1^{(i)}, e_2^{(i)}\}_{i \in [M]}$ to $\mathcal{V}$.

Verify phase:

$\mathcal{V}$ checks:

1. $MT.Eval.Verify(rt_{E1}^{(i)}, \Pi_{E1}^{(i)}, e_1^{(i)}, idx) = 1$, for $i \in [M]$.
2. $MT.Eval.Verify(rt_{E2}^{(i)}, \Pi_{E2}^{(i)}, e_2^{(i)}, idx) = 1$, for $i \in [M]$.
3. $e_1 = \sum_{i \in [M]} e_1^{(i)}$ and $e_2 = \sum_{i \in [M]} e_2^{(i)}$.

**Polynomial commitment** We now describe the procedure for distributed polynomial commitment to vectors $E_1, F_1, E_2, F_2$, where the prover claims to the verifier that $\tilde{E}_1(x) = v_{E1}$, $\tilde{F}_1(x) = v_{F1}$, $\tilde{E}_2(x) = v_{E2}$, $\tilde{F}_2(x) = v_{F2}$.The process for $\tilde{E}_1$ and $\tilde{E}_2$ is similar to the Merkle commitment For $E_1$ and $E_2$. Each sub-prover $\mathcal{P}_i$ constructs a commitment to $\tilde{E}_1^{(i)}, \tilde{E}_2^{(i)}$, denoted $C_{E1}^{(i)}, C_{E2}^{(i)}$, and sends them to the master prover $\mathcal{P}_0$. Then $\mathcal{P}_0$ forwards the set of all commitments $\{C_{E1}^{(i)}, C_{E2}^{(i)}\}_{i \in [M]}$ to the verifier $\mathcal{V}$. During the evaluation phase, each $\mathcal{P}_i$ supplies the proofs $\pi_{E1}^{(i)}$ and $\pi_{E2}^{(i)}$ to $\mathcal{P}_0$, which forwards the set of all proofs $\{\pi_{E1}^{(i)}, \pi_{E2}^{(i)}\}_{i \in [M]}$ to the verifier $\mathcal{V}$. $\mathcal{V}$ individually checks each of these proofs, and finally verifies that $v_{E1} = \sum_{i \in [M]} v_{E1}^{(i)}$, $v_E = \sum_{i \in [M]} v_{E2}^{(i)}$. The detailed constructions for committing to the polynomial $\tilde{E}_1, \tilde{E}_2$ are presented in Protocol 8. The same approach applies to vectors $\tilde{F}_1, \tilde{F}_2$, with the corresponding details provided in Protocol 9.

**Protocol 8: Distributed Polynomial Commitment to $\tilde{E}_1$ and $\tilde{E}_2$.** $\mathcal{P}_0$ claims to $\mathcal{V}$ that $\tilde{E}_1(x) = v_{E1}$, $\tilde{E}_2(x) = v_{E2}$. $\mathcal{P}_i$ holds sub-matrix $M_f^{(i)} \in \mathbb{F}^{b \times \frac{N}{B}}$ and $\mathbf{a}^{(i)} \in \mathbb{F}^b$, where $M_f^{(i)}[k] = M_f[ib + k:], \mathbf{a}^{(i)}[k] = \mathbf{a}[ib + k]$, and $b = \frac{B}{M}$. Let $\beta^{(i)} \in \mathbb{F}^b$ where $\beta^{(i)}[k] = \mathsf{eq}(\mathbf{u}_1, bin(ib + k))$. $PC = (Gen, Commit, Open, Eval)$ is the underlying polynomial commitment. $Enc : \mathbb{F}^{\frac{N}{B}} \to \mathbb{F}^{\frac{cN}{B}}$ is the map of the encoding algorithm for the linear

code in Brakedown [12].

Commit phase:

1. $\mathcal{P}_i$ computes $M_f'^{(i)} \in \mathbb{F}^{b \times N/B}$ such that $M_f'^{(i)}[k:] = Enc(M_f^{(i)}[k:])$, for $k \in [b]$. Then $\mathcal{P}_i$ computes $E_1^{(i)}, E_2^{(i)}$, where $E_1^{(i)} = \sum_{k\in[b]} \mathbf{a}^{(i)}[k] M_f'^{(i)}[k]$, and $E_2^{(i)} = \sum_{k\in[b]} \beta^{(i)}[k] M_f'^{(i)}[k]$.
2. $\mathcal{P}_i$ generates the commitments $C_{E1}^{(i)} \leftarrow PC.Commit(\tilde{E}_1^{(i)})$ and $C_{E2}^{(i)} \leftarrow PC.Commit(\tilde{E}_2^{(i)})$. Then $\mathcal{P}_i$ sends $C_{E1}^{(i)}, C_{E2}^{(i)}$ to $\mathcal{P}_0$.
3. $\mathcal{P}_0$ outputs $\{C_{E1}^{(i)}, C_{E2}^{(i)}\}_{i\in[M]}$.

Eval phase:

1. $\mathcal{P}_i$ sends the proofs $\pi_{E1}^{(i)} \leftarrow PC.Eval.Prove(\tilde{E}_1^{(i)}, x, v_{E1}^{(i)})$, $\pi_{E2}^{(i)} \leftarrow PC.Eval.Prove(\tilde{E}_2^{(i)}, x, v_{E2}^{(i)})$ to $\mathcal{P}_0$, and $\mathcal{P}_0$ sends $\{\pi_{E1}^{(i)}, \pi_{E2}^{(i)}\}_{i\in[M]}$ and the claimed values $\{v_{E1}^{(i)}, v_{E2}^{(i)}\}_{i\in[M]}$ to $\mathcal{V}$.
2. $\mathcal{V}$ checks:
   (a) $PC.Eval.Verify(C_{E1}^{(i)}, \pi_{E1}^{(i)}, v_{E1}^{(i)}, x) = 1$, for $i \in [M]$.
   (b) $PC.Eval.Verify(C_{E2}^{(i)}, \pi_{E2}^{(i)}, v_{E2}^{(i)}, x) = 1$, for $i \in [M]$.
   (c) $v_{E1} = \sum_{i\in[M]} v_{E1}^{(i)}$, $v_E = \sum_{i\in[M]} v_{E2}^{(i)}$.

**Protocol 9: Distributed Polynomial Commitment to $\tilde{F}_1$ and $\tilde{F}_2$.** $\mathcal{P}_0$ claims to $\mathcal{V}$ that $\tilde{F}_1(x) = v_{F1}$, $\tilde{F}_2(x) = v_{F2}$. $\mathcal{P}_i$ holds sub-matrix $M_f^{(i)} \in \mathbb{F}^{b\times\frac{N}{B}}$ and $\mathbf{a}^{(i)} \in \mathbb{F}^b$, where $M_f^{(i)}[k] = M_f[ib+k:], \mathbf{a}^{(i)}[k] = \mathbf{a}[ib+k]$, and $b = \frac{B}{M}$. Let $\beta^{(i)} \in \mathbb{F}^b$ where $\beta^{(i)}[k] = \mathsf{eq}(\mathbf{u}_1, bin(ib+k))$. $PC = (Gen, Commit, Open, Eval)$ is the underlying polynomial commitment. $Enc : \mathbb{F}^{\frac{N}{B}} \to \mathbb{F}^{\frac{cN}{B}}$ is the map of the encoding algorithm for the linear code in Brakedown [12].

Commit phase:

1. $\mathcal{P}_i$ computes $F_1^{(i)}, F_2^{(i)}$, where $F_1^{(i)} = \sum_{k\in[b]} \mathbf{a}^{(i)}[k] M_f^{(i)}[k]$, and $F_2^{(i)} = \sum_{k\in[b]} \beta^{(i)}[k] M_f^{(i)}[k]$.
2. $\mathcal{P}_i$ generates the commitments $C_{F1}^{(i)} \leftarrow PC.Commit(\tilde{F}_1^{(i)})$ and $C_{F2}^{(i)} \leftarrow PC.Commit(\tilde{F}_2^{(i)})$. Then $\mathcal{P}_i$ sends $C_{F1}^{(i)}, C_{F2}^{(i)}$ to $\mathcal{P}_0$.
3. $\mathcal{P}_0$ outputs $\{C_{F1}^{(i)}, C_{F2}^{(i)}\}_{i\in[M]}$.

Eval phase:

1. $\mathcal{P}_i$ sends the proofs $\pi_{F1}^{(i)} \leftarrow PC.Eval.Prove(\tilde{F}_1^{(i)}, x, v_{F1}^{(i)})$, $\pi_{F2}^{(i)} \leftarrow PC.Eval.Prove(\tilde{F}_2^{(i)}, x, v_{F2}^{(i)})$ to $\mathcal{P}_0$, and $\mathcal{P}_0$ sends $\{\pi_{F1}^{(i)}, \pi_{F2}^{(i)}\}_{i\in[M]}$ and the claimed values $\{v_{F1}^{(i)}, v_{F2}^{(i)}\}_{i\in[M]}$ to $\mathcal{V}$.
2. For $i \in [M]$, $\mathcal{V}$ checks:
   (a) $PC.Eval.Verify(C_{F1}^{(i)}, \pi_{F1}^{(i)}, v_{F1}^{(i)}, x) = 1$.

(b) $PC.Eval.Verify(C_{F2}^{(i)}, \pi_{F2}^{(i)}, v_{F2}^{(i)}, x) = 1$.
(c) $v_{F1} = \sum_{i\in[M]} v_{F1}^{(i)}$, $v_{F2} = \sum_{i\in[M]} v_{F2}^{(i)}$.

### 4.3 Distributed Proof of Relation $\mathcal{R}$

We now explain how the sub-provers collectively convince the verifier of the relation $\mathcal{R}$ in step 9 of protocol 6. Since the proof for relation $\mathcal{R}$ comprises the proofs of encoding algorithms $E_1 = Enc(F_1), E_2 = Enc(F_2)$, we consider that the provers claim to the verifier that $E = Enc(F)$ for the vectors $E \in \mathbb{F}^{\frac{cN}{B}}$ and $F \in \mathbb{F}^{\frac{N}{B}}$, while both parties hold the polynomial commitment of $\tilde{E}$ and $\tilde{F}$, denoted $C_E$ and $C_F$. To prove that $E = Enc(F)$, it suffices to show that:(1) $E$ is a valid codeword, specifically, there exists a parity check matrix $H \in \mathbb{F}^{\log\frac{cN}{B}\times\log\frac{(c-1)N}{B}}$ for the coding, where $c$ is the reciprocal of the rate of the code, such that $E \cdot H = \mathbf{0}$;(2) The first $\log\frac{N}{B}$ entries of $E$ are exactly equal to vector $F$(the encoding is systematic). As shown in [20], the matrix $H$ is sparse and thus has $O(\frac{N}{B})$ non-zero entries. Each sub-prover holds a split of $H$ and the $i$-th sub-prover computes $\tilde{H}_{\mathbf{u}}(x, bin(i))$ efficiently.

Statement (1) can be reduced to a sumcheck relation $\sum_{x\in\{0,1\}^{\log\frac{cN}{B}}} \tilde{E}(x) \cdot \tilde{H}_{\mathbf{u}}(x) = 0$. Let $Q(x) = \tilde{E}(x) \cdot \tilde{H}_{\mathbf{u}}(x)$. As in the sumcheck procedure, the prover is required to send $Q_k(X_k) = \sum_{x\in\{0,1\}^{\log\frac{cN}{B}-k}} Q(r_1, \ldots, r_{k-1}, X_k, x)$ to the verifier in the $k$-th round, where $1 \le k \le \log\frac{cN}{B}$ and $r_k$ is the randomness sent by the verifier in the $k$-th round. In our distributed proof, each sub-prover $\mathcal{P}_i$ holds $E^{(i)} = \sum_{k\in[b]} \mathbf{a}^{(i)}[k]Enc(M_f^{(i)})[k]$ and the parity check matrix $H \in \mathbb{F}^{\frac{cN}{B}\times\frac{(c-1)N}{B}}$ for the linear code in Brakedown [12]for the massage of length $\frac{N}{B}$, such that $\mathcal{P}_i$ computes the values of $\tilde{E}_k^{(i)}(X_k, x) = \tilde{E}^{(i)}(r_1, \ldots, r_{k-1}, X_k, x)$ and $\tilde{H}_{\mathbf{u},k}(X_k, x) = \tilde{H}_{\mathbf{u}}(r_1, \ldots, r_{k-1}, X_k, x)$ for all $x \in \{0,1\}^{\log\frac{cN}{B}-k}$ in the $k$-th round. Then $\mathcal{P}_i$ computes $Q_k^{(i)}(X_k) \leftarrow \sum_{x\in\{0,1\}^{\log\frac{cN}{B}-k}} \tilde{E}_k^{(i)}(X_k, x) \cdot \tilde{H}_{\mathbf{u},k}(X_k, x)$ and $\mathcal{P}_0$ gets $Q_k(X_k) = \sum_{i\in[M]} Q_k^{(i)}(X_k)$, as

$$\begin{aligned}
\sum_{i\in[M]} Q_k^{(i)}(X_k) &= \sum_{i\in[M]} \sum_{x\in\{0,1\}^{\log\frac{cN}{B}-k}} \tilde{E}_k^{(i)}(X_k, x) \cdot \tilde{H}_{\mathbf{u},k}(X_k, x) \\
&= \sum_{x\in\{0,1\}^{\log\frac{cN}{B}-k}} (\sum_{i\in[M]} \tilde{E}_k^{(i)}(X_k, x)) \cdot \tilde{H}_{\mathbf{u},k}(X_k, x) \\
&= \sum_{x\in\{0,1\}^{\log\frac{cN}{B}-k}} \tilde{E}_k(X_k, x) \cdot \tilde{H}_{\mathbf{u},k} = Q_k(X_k)
\end{aligned}$$

To prove statement (2), it is equal to proving that $\tilde{E}(x, 0^{\log c}) = \tilde{F}(x)$, for any $x \in \{0,1\}^{\log\frac{N}{B}}$. The verifier samples a random $\mathbf{u}' \in \mathbb{F}^{\log\frac{N}{B}}$, and verify that $\tilde{E}(\mathbf{u}', 0^{\log c}) = \tilde{F}(\mathbf{u}')$. The soundness is guaranteed by the Schwarz-Zippel lemma.

At the end of the protocol, the verifier "query" the value of $\tilde{E}, \tilde{F}$ and $\tilde{H}_{\mathbf{u}}$. The query of $\tilde{E}, \tilde{F}$ is instantiated with the distributed polynomial commitment construction(Protocol 8 and 9) tailored in distributed Brakedown. The query of $\tilde{H}_{\mathbf{u}}$ is instantiated with the BaseFold protocol. We present the full construction of the distributed proof of encoding algorithm in Protocol 10.

**Protocol 10: Distributed Proof of Encoding Algorithm**

$\mathcal{P}_0$ claims to $\mathcal{V}$ that $E = Enc(F)$, where $E \in \mathbb{F}^{\frac{cN}{B}}, F \in \mathbb{F}^{\frac{N}{B}}$ and $Enc : \mathbb{F}^{\frac{N}{B}} \to \mathbb{F}^{\frac{cN}{B}}$ is the map of the encoding algorithm for the linear code in Brakedown [12]. $\mathcal{P}_i$ holds $\tilde{E}^{(i)}(x)$ and the parity check matrix $H \in \mathbb{F}^{\frac{cN}{B} \times \frac{(c-1)N}{B}}$ for the linear code for the massage of length $\frac{N}{B}$.

1. $\mathcal{V}$ samples a random vector $\mathbf{u} \in \mathbb{F}^{\log \frac{(c-1)N}{B}}$ and sends $\mathbf{u}$ to $\mathcal{P}_0$, and $\mathcal{P}_0$ transmits $\mathbf{u}$ to $\mathcal{P}_i$.
2. Each $\mathcal{P}_i$ computes $\tilde{H}_{\mathbf{u}}(b) = \sum_{t \in \{0,1\}^{\log \frac{(c-1)N}{B}}} \tilde{H}(b,t) \cdot \mathsf{eq}(t, \mathbf{u})$ for $b \in \{0,1\}^{\log \frac{cN}{B}}$, where $\tilde{H}_{\mathbf{u}}(b) = \tilde{H}(b, \mathbf{u})$.
3. $\mathcal{P}_0, \ldots, \mathcal{P}_{M-1}$ and $\mathcal{V}$ perform the sumcheck protocol for $\sum_{b \in \{0,1\}^{\log \frac{cN}{B}}} \tilde{E}(b) \cdot \tilde{H}_{\mathbf{u}}(b) = 0$. Specifically, in the $k$-th round, where $1 \le k \le \log \frac{cN}{B}$:
   (a) Each $\mathcal{P}_i$ locally computes the polynomials $\tilde{E}_k^{(i)}(X_k, x)$ and $\tilde{H}_{\mathbf{u},k}(X_k, x)$ for $x \in \{0,1\}^{\log \frac{cN}{B} - k}$, where:
   $\tilde{E}_k^{(i)}(X_k, x) := \tilde{E}^{(i)}(r_1, \ldots, r_{k-1}, X_k, x)$,
   $\tilde{H}_{\mathbf{u},k}(X_k, x) := \tilde{H}_{\mathbf{u}}(r_1, \ldots, r_{k-1}, X_k, x)$.
   (b) $\mathcal{P}_i$ sends $Q_k^{(i)}(X_k) \leftarrow \sum_{x \in \{0,1\}^{\log \frac{cN}{B} - k}} \tilde{E}_k^{(i)}(X_k, x) \cdot \tilde{H}_{\mathbf{u},k}(X_k, x)$ to $\mathcal{P}_0$, and then $\mathcal{P}_0$ sends $Q_k(X_k) \leftarrow \sum_{i \in [M]} Q_k^{(i)}(X_k)$ to $\mathcal{V}$.
   (c) $\mathcal{V}$ checks if $q_{k-1} = Q_k(0) + Q_k(1)$(Set $q_0 = 0$). If the check passes, $\mathcal{V}$ sends a random $r_k \in \mathbb{F}$ to $\mathcal{P}_0$, and set $q_k \leftarrow Q_k(r_k)$. $\mathcal{P}_0$ transmits $r_k$ to $\mathcal{P}_i$.
4. $\mathcal{P}_0, \ldots, \mathcal{P}_{M-1}$ and $\mathcal{V}$ perform the evaluation proof protocol for $\tilde{H}_{\mathbf{u}}(\mathbf{r}) = q_1'$ and $\tilde{E}(\mathbf{r}) = q_2'$, where $\mathbf{r} = (r_1, \ldots, r_{\log \frac{cN}{B}})$, and check $q_{\log \frac{cN}{B}} = q_1' \cdot q_2'$. For the relation $\tilde{H}_{\mathbf{u}}(\mathbf{r}) = q_1'$, $\mathcal{P}_0, \ldots, \mathcal{P}_{M-1}$ and $\mathcal{V}$ perform the distributed BaseFold protocol in [31]. For the relation $\tilde{E}(\mathbf{r}) = q_2'$, $\mathcal{P}_0, \ldots, \mathcal{P}_{M-1}$ and $\mathcal{V}$ perform the evaluate phase of the distributed polynomial commitment to $\tilde{E}$(Protocol 8).
5. $\mathcal{V}$ samples a random vector $\mathbf{u}' \in \mathbb{F}^{\log \frac{N}{B}}$. $\mathcal{P}_0, \ldots, \mathcal{P}_{M-1}$ and $\mathcal{V}$ perform the evaluate phase of the distributed polynomial commitment to $\tilde{E}$ and $\tilde{F}$(Protocol 8 and 9), and verify that $\tilde{E}(\mathbf{u}', 0^{\log c}) = \tilde{F}(\mathbf{u}')$.

### 4.4 Distributed Brakedown

We now integrate the above components to construct the distributed version of Brakedown, where the matrix $M_f$ is partitioned row-wise and evenly dis-

tributed among the sub-provers, and each sub-prover $\mathcal{P}_i$ holds sub-matrix $M_f^{(i)}$. The protocol incorporates the distributed polynomial commitments to vectors $E_1, F_1, E_2, F_2$, Merkle hash commitments to vectors $E_1, E_2$(Subsection 4.2), and the distributed proof for relation $\mathcal{R}$(Subsection 4.3). In the commitment phase, $\mathcal{P}_i$ hashes the column vectors of $M_f^{(i)}$ and constructs a Merkle tree root $C^{(i)}$ , and the commitment is the set $\{C^{(i)}\}_{i\in[M]}$. To open the commitment, each $\mathcal{P}_i$ provides a proof that its root correctly commits to the corresponding sub-matrix. The verifier checks all proofs individually, and the overall verification succeeds only if every proof is valid. Moreover, we note that $F_2$ is opened once at point $r_2$. Since the procedure for opening $\tilde{F}_2$ is entirely analogous to that for opening $\tilde{F}_2$ in Protocol 9, we omit a detailed description here. The full construction is presented in Protocol 11.

**Protocol 11: Distributed Brakedown**

$\mathcal{P}_0$ claims $f(\mathbf{s}) = v$ to $\mathcal{V}$, where $f \in \mathcal{F}^{(\leq 1)}_{\log N}$, and $\mathbf{s} = \mathbf{s}_1||\mathbf{s}_2$, $\mathbf{s}_1 \in \mathbb{F}^{\log B}$, $\mathbf{s}_2 \in \mathbb{F}^{\log \frac{N}{B}}$. Let $Enc : \mathbb{F}^{\frac{N}{B}} \to \mathbb{F}^{\frac{cN}{B}}$ be the map of the encoding algorithm for the linear code in Brakedown [12]. $\mathcal{P}_i$ holds sub-matrix $M_f^{(i)} \in \mathbb{F}^{b\times \frac{N}{B}}$, where $M_f^{(i)}[k] = M_f[ib + k :]$. Let $\beta^{(i)} \in \mathbb{F}^b$ where $\beta^{(i)}[k] = \mathsf{eq}(\mathbf{s}_1, bin(ib + k))$. $PC = (Gen, Commit, Open, Eval)$ is the underlying polynomial commitment. $MT = (Commit, Eval)$ is Merkle commitment. Let $H(\cdot) : \mathbb{F}^* \to \mathbb{F}$ be a random oracle.

Commit phase:

$\mathcal{P}_i$ computes $M_f'^{(i)} \in \mathbb{F}^{b\times N/B}$ such that $M_f'^{(i)}[k :] = Enc(M_f^{(i)}[k :])$, for $k \in [b]$. Let $\mathbf{h}^{(i)} \in \mathbb{F}^{\frac{N}{B}}$ be a vector such that $\mathbf{h}^{(i)}[k] \leftarrow H(M_f'^{(i)}[: k])$, for each $k \in [\frac{N}{B}]$. $\mathcal{P}_i$ sends $C^{(i)} \leftarrow MT.Commit(\mathbf{h}^{(i)})$ to $\mathcal{P}_0$. $\mathcal{P}_0$ outputs $\{C^{(i)}\}_{i\in[M]}$.

Eval phase:

1. $\mathcal{V}$ samples a random vector $\mathbf{a} \in \mathbb{F}^B$, and $\mathcal{P}_0$ sends the vector $\mathbf{a}^{(i)}$ to $\mathcal{P}_i$, where $\mathbf{a}^{(i)} = \mathbf{a}[ib : (i + 1)b - 1]$.
2. $\mathcal{P}_i$ performs the commit phase of the of the distributed polynomial commitment to $\tilde{E}_1^{(i)}$, $\tilde{F}_1^{(i)}$, $\tilde{E}_2^{(i)}$ and $\tilde{F}_2^{(i)}$ using $Commit.PC$, where $E_1^{(i)} = Enc(F_1^{(i)})$, $F_1^{(i)} = \sum_{k\in[b]} \mathbf{a}^{(i)}[k]M_f^{(i)}[k]$, $E_2^{(i)} = Enc(F_2^{(i)})$, $F_2^{(i)} = \sum_{k\in[b]} \beta^{(i)}[k]M_f[k]$ for $i \in [M]$. Then $\mathcal{P}_i$ sends the corresponding commitments $C_{E1}^{(i)}, C_{F1}^{(i)}, C_{E2}^{(i)}, C_{F2}^{(i)}$ to $\mathcal{P}_0$. $\mathcal{P}_0$ sends $\{C_{E1}^{(i)}, C_{F1}^{(i)}, C_{E2}^{(i)}, C_{F2}^{(i)}\}_{i\in[M]}$ to $\mathcal{V}$.
3. $\mathcal{P}_i$ sends the merkle-tree roots $rt_1^{(i)} \leftarrow MT.Commit(E_1^{(i)})$ and $rt_2^{(i)} \leftarrow MT.Commit(E_2^{(i)})$ to $\mathcal{P}_0$. $\mathcal{P}_0$ sends $\{rt_1^{(i)}, rt_2^{(i)}\}_{i\in[M]}$ to $\mathcal{V}$.
4. $\mathcal{V}$ sends the column indexes $\mathcal{I} = \{idx_1, \ldots, idx_\ell\}$ to $\mathcal{P}_0$. $\mathcal{P}_0$ transmits $\mathcal{I}$ to $\mathcal{P}_i$.

5. $\mathcal{P}_i$ chooses the columns $R^{(i)} \in \mathbb{F}^{\ell \times b}$ where $R^{(i)}[j :] = M_f^{(i)}[: idx_j]$ for all $idx_j \in \mathcal{I}$, and generates the proof $\pi_{i,j} \leftarrow MT.Eval.Prove(\mathbf{h}^{(i)}, idx_j, \overline{\mathbf{h}}^{(i)}[j])$ for $j \in [\ell]$, where $\overline{\mathbf{h}}^{(i)} \in \mathbb{F}^{\ell}$ and $\overline{\mathbf{h}}^{(i)}[j] = H(R^{(i)}[j :])$ for $i \in [M], j \in [\ell]$. $\mathcal{P}_i$ sends $\{\pi_{i,j}\}_{j\in[\ell]}$ and $\overline{\mathbf{h}}^{(i)}$ to $\mathcal{P}_0$, and $\mathcal{P}_0$ sends $\{\pi_{i,j}\}_{i\in[M],j\in[\ell]}$ and $\{\overline{\mathbf{h}}^{(i)}\}_{i\in[M]}$ to $\mathcal{V}$.
6. $\mathcal{P}_i$ computes the vectors $\mathbf{y}_1^{(i)}, \mathbf{y}_2^{(i)} \in \mathbb{F}^{\ell}$, such that $\mathbf{y}_1^{(i)}[k] = \sum_{j\in[b]} R^{(i)}[k,j] \cdot \mathbf{a}^{(i)}[j], \mathbf{y}_2^{(i)}[k] = \sum_{j\in[b]} R^{(i)}[k,j] \cdot \beta^{(i)}[j]$, for $k \in [\ell]$. $\mathcal{P}_0$ computes $\mathbf{y}_1 \leftarrow \sum_{i\in[M]} \mathbf{y}_1^{(i)}, \mathbf{y}_2 \leftarrow \sum_{i\in[M]} \mathbf{y}_2^{(i)}$ and sends $\mathbf{y}_1, \mathbf{y}_2$ to $\mathcal{V}$.
7. $\mathcal{P}_i$ sends $\pi_{F2}^{(i)}$ and the claimed value $v_{F2}^{(i)}$ to $\mathcal{P}_0$ , where $\pi_{F2}^{(i)} \leftarrow PC.Eval.Prove(\tilde{F}_2^{(i)}, \mathbf{s}_2, v_{F2}^{(i)})$ to $\mathcal{P}_0$. $\mathcal{P}_0$ sends $\{v_{F2}^{(i)}, \pi_{F2}^{(i)}\}_{i\in[M]}$ to $\mathcal{V}$.
8. $\mathcal{P}_i$ generates the proof $\{\Pi_{1,j}^{(i)}, \Pi_{2,j}^{(i)}\}_{j\in[\ell]}$ and the claimed values $\{e_{1,j}^{(i)}, e_{2,j}^{(i)}\}_{j\in[\ell]}$ to $\mathcal{P}_0$, where $\Pi_{1,j}^{(i)} = MT.Eval.Prove(E_1^{(i)}, idx_j, e_{1,j}^{(i)})$, $\Pi_{2,j}^{(i)} = MT.Eval.Prove(E_2^{(i)}, idx_j, e_{2,j}^{(i)})$, for $i \in [M], j \in [\ell]$. $\mathcal{P}_0$ sends $\{\Pi_{1,j}^{(i)}, \Pi_{2,j}^{(i)}\}_{i\in[M],j\in[\ell]}$, and $\{e_{1,j}^{(i)}, e_{2,j}^{(i)}\}_{i\in[M],j\in[\ell]}$ to $\mathcal{V}$.
9. $\mathcal{P}_0, \ldots, \mathcal{P}_{M-1}$ and $\mathcal{V}$ perform Protocol 10 for the ralation $E_1 = Enc(F_1), E_2 = Enc(F_2)$ and $\mathcal{P}_0$ generates the proof $\pi_r$ for the relation $\mathcal{R}$. $\mathcal{P}_0$ sends $\pi_r$ to $\mathcal{V}$.
10. $\mathcal{V}$ checks:
    (a) $MT.Eval.Verify(C^{(i)}, \pi_{i,j}, \overline{\mathbf{h}}^{(i)}[j], idx_j) = 1$, for $i \in [M], j \in [\ell]$.
    (b) $\mathbf{y}_1[k] = \sum_{i\in[M]} \sum_{j\in[B]} R^{(i)}[k,j]\mathbf{a}[ib+j]$, and
    $\mathbf{y}_2[k] = \sum_{i\in[M]} \sum_{j\in[B]} R^{(i)}[k,j]\beta^{(i)}[j]$, for $k \in [\ell]$.
    (c) $PC.Eval.Verify(C_{F2}^{(i)}, \pi_{F_2}^{(i)}, v_{F_2}^{(i)}, \mathbf{s}_2) = 1$, for $i \in [M]$, and
    $\sum_{i\in[M]} v_{F2}^{(i)} = v$.
    (d) $\mathbf{y}_1[j] = \sum_{i\in[M]} e_{1,j}, \mathbf{y}_2[j] = \sum_{i\in[M]} e_{2,j}$, and
    $MT.Eval.Verify(rt_1^{(i)}, \Pi_{1,j}^{(i)}, e_{1,j}^{(i)}, idx_j) = 1$,
    $MT.Eval.Verify(rt_2^{(i)}, \Pi_{2,j}^{(i)}, e_{2,j}^{(i)}, idx_j) = 1$, for $i \in [M], j \in [\ell]$.
    (e) the relation $\mathcal{R}$ is valid with the proof $\pi_r$.

### 4.5 Security and Complexity Analysis

**Security** We analyze the security of our distributed Brakedown protocol(Protocol 11). As in the Section 3, we analyze security under the single-prover setting. We start our analysis by analyzing the security of two core components: polynomial commitment to $\tilde{E}_1, \tilde{F}_1, \tilde{E}_2, \tilde{F}_2$, and proof for relation $\mathcal{R}$. We then demonstrate the overall security of the distributed Brakedown protocol.

*Security of Polynomial Commitment to $\tilde{E}_1, \tilde{F}_1, \tilde{E}_2, \tilde{F}_2$.* We analyze completeness and knowledge soundness of distributed polynomial commitment to $\tilde{E}_1, \tilde{E}_2$(Protocol 8) and $\tilde{F}_1, \tilde{F}_2$(Protocol 9). First, we prove the following lemma:

**Lemma 3.** *Let $m \in \mathbb{N}$ and $m = O(1)$. Given polynomial commitments to each of the polynomials $f_1, \ldots, f_m$, we can construct a commitment to the linearly combined polynomial $\sum_{i \in m} a_i f_i$ as follows:*
*The prover claims $f(\boldsymbol{x}) = v$ to the verifier, where $v = \sum_{i \in m} a_i f_i$.*
***Gen phase**: Generate the public parameters for the polynomial commitment to $f_1, \ldots, f_m$.*
***Open phase**: Open the the commitments to $f_1, \ldots, f_m$ and get $f'_1, \ldots, f'_m$, output $\sum_{i \in [m]} a_i f'_i$.*
***Commit phase**: The prover sends the commitments to $f_1, \ldots, f_m$ to the verifier.*
***Eval phase**: The prover sends the proof for $f_i(\boldsymbol{x}) = v_i$, for $i \in [m]$ to the verifier, and the verifier checks all proofs are valid, and $v = \sum_{i \in [m]} a_i v_i$.*

*Proof.* We demonstrate that if the underlying commitments to the polynomials $f_1, \ldots, f_m$ satisfy completeness, binding, and knowledge soundness, then the resulting commitment to the linear combination $\sum_{i \in m} a_i f_i$ also satisfies these properties.

The completeness is from the completeness from the polynomials $f_1, \ldots, f_m$, and if $f_i(\mathbf{x}) = v_i$, for $i \in [m]$, then $v = \sum_{i \in [m]} a_i f_i(\mathbf{x}) = a_i v_i$.

To prove the binding property, we open the commitment of $f_i$ twice, and get the polynomials $f_{i,1}$ and $f_{i,2}$, for $i \in [m]$. By the binding property of the underlying commitments to $f_1, \ldots, f_m$, the probability of $f_{i,1} \neq f_{i,2}$ is negligible in $\lambda$ for *some* $i \in [m]$. Therefore, the probability that $f_{i,1} = f_{i,2}$ holds for all $i \in [m]$ is at least $(1 - \mathsf{negl}(\lambda))^m$. Using the inequality $(1-\epsilon)^m \geq 1 - m\epsilon$ for small $\epsilon$, this probability is bounded below by $1 - m \cdot \mathsf{negl}(\lambda)$. When $m = O(1)$, the deviation $m \cdot \mathsf{negl}(\lambda)$ remains negligible. Hence, the probability that the two linear combinations differ, i.e.,$\sum_{i \in [m]} a_i f_{i,1} \neq \sum_{i \in [m]} a_i f_{i,2}$, is also negligible in $\lambda$, which confirms that the binding property is satisfied.

To prove the knowledge soundness property, we construct an extractor that utilizes the extractors of the underlying commitments, which outputs $f_1^*, \ldots, f_m^*$, and outputs $f^* = \sum_{i \in [m]} a_i f_i^*$. By the knowledge soundness property of the underlying commitments to $f_1, \ldots, f_m$, the probability of $f_i^*(\mathbf{x}) = v_i$ for *some* $i \in [m]$ is $1 - \mathsf{negl}(\lambda)$. Therefore, the probability that $f_i^*(\mathbf{x}) = v_i$ for *all* $i \in [m]$ is $(1 - \mathsf{negl}(\lambda))^m$. Using the inequality $(1-\epsilon)^m \geq 1 - m\epsilon$ for small $\epsilon$, this probability is bounded below by $1 - m \cdot \mathsf{negl}(\lambda)$. When $m = O(1)$, the deviation $m \cdot \mathsf{negl}(\lambda)$ remains negligible. Hence, the probability of $f^*(\mathbf{x}) \neq v$ is negligible in $\lambda$, which confirms that the knowledge soundness property is satisfied. □

We can achieve linearly combined polynomial commitment to $\tilde{E}_1, \tilde{E}_2$ from the polynomial commitment to $\{\tilde{E}_1^{(i)}\}_{i \in [M]}, \{\tilde{E}_2^{(i)}\}_{i \in [M]}$ respectively, and then achieve linearly combined polynomial commitment to $\tilde{E}$ from the polynomial commitment to $\tilde{E}_1$ and $\tilde{E}_2$. The same procedure applies to $\tilde{F}_1, \tilde{F}_2$. By Lemma 3, we prove that the polynomial commitment to $\tilde{E}_1, \tilde{E}_2, \tilde{F}_1, \tilde{F}_2$ satisfies completeness, binding, and knowledge soundness, provided that the underlying $PC$ already satisfies these properties.

*Security of proof for relation $\mathcal{R}$* The proof for relation $\mathcal{R}$ includes the proofs of the two encodings $E_1 = Enc(F_1)$ and $E_2 = Enc(F_2)$. Therefore, it suffices

to analyze the security of the proof of the encoding algorithm as specified in Protocol 10.

The completeness of the protocol follows directly from the completeness of the underlying sumcheck protocol, the Basefold protocol, and the commitments to $\tilde{E}$ and $\tilde{F}$. The soundness property is established as follows.

Let $\mathcal{A}_{Enc}$ be a PPT adversary that produces an accepting transcript when interacting with the verifier over the protocol proving the relation $\mathcal{R}$, hence we construct our extractor $\mathcal{E}_{Enc}$ as follows: (1) $\mathcal{E}_R$ uses $\mathcal{A}_{Enc}$ to construct $\mathcal{A}_p, \mathcal{A}_E$, where $\mathcal{A}_p, \mathcal{A}_E$ is the PPT adversary that produces an accepting transcript when interacting with the verifier over the Basefold protocol, the commitment to $\tilde{E}$, respectively. (2) $\mathcal{E}_{Enc}$ invokes the corresponding extractors $\mathcal{E}_p, \mathcal{E}_E$ to obtain the $\tilde{H}_{\mathbf{u}}, \tilde{E}$ respectively, where $\tilde{H}_{\mathbf{u}}(\mathbf{r}) = q_1', \tilde{E}(\mathbf{r}) = q_2'$.(3) $\mathcal{E}_R$ uses $\tilde{H}_{\mathbf{u}}, \tilde{E}$ to construct $\mathcal{A}_{sc}$, which produces an accepting transcript when interacting with the verifier over the sumcheck protocol. (4) $\mathcal{E}_{Enc}$ invokes $\mathcal{E}_{sc}$ to get the $\tilde{H}_{\mathbf{u}}$ and $\tilde{E}$, where $\sum_{b \in \{0,1\}^{\log \frac{cN}{B}}} \tilde{E}(b) \cdot \tilde{H}_{\mathbf{u}}(b) = 0$. (5) $\mathcal{E}_{Enc}$ uses $\mathcal{A}_{Enc}$ to construct $\mathcal{A}_E', \mathcal{A}_F'$, where $\mathcal{A}_E', \mathcal{A}_F'$ is the PPT adversary that produces an accepting transcript when interacting with the verifier over the commitment to $\tilde{E}$, the commitment to $\tilde{F}$ in step 5, respectively. (6) $\mathcal{E}_{Enc}$ invokes the corresponding extractor $\mathcal{E}_E', \mathcal{E}_F'$ to obtain $\tilde{E}', \tilde{F}'$.(7) If $\tilde{E} \neq \tilde{E}'$, abort, else output $\tilde{E}, \tilde{F}'$.

By the soundness of the Basefold protocol, the sumcheck protocol, and the commitment to $\tilde{E}$ and $\tilde{F}$, $\mathcal{E}_p, \mathcal{E}_{sc}, \mathcal{E}_E, \mathcal{E}_E', \mathcal{E}_F'$ outputs the corresponding witness with probability $1 - \mathsf{negl}(\lambda)$. By the binding property of the polynomial commitment to $\tilde{E}$, the probability of $\tilde{E} \neq \tilde{E}'$ is negligible in $\lambda$. Consequently, $\mathcal{E}_R$ outputs $\tilde{H}, \tilde{E}, \tilde{F}'$ satisfying relation $\mathcal{R}$ with $1 - \mathsf{negl}(\lambda)$ probability. Furthermore, $\mathcal{E}_p, \mathcal{E}_{sc}, \mathcal{E}_E, \mathcal{E}_E', \mathcal{E}_F'$ all run in polynomial time, thus $\mathcal{E}_{Enc}$ also runs in polynomial time. Therefore, the proof of the encoding algorithm in Protocol 10 satisfies knowledge soundness.

*Security of our PCS* We now have all the components required to establish the completeness, binding, and knowledge soundness of our distributed polynomial commitment scheme. The completeness property follows directly from the completeness of the underlying Brakedown construction, as well as the completeness of the commitments to $\tilde{E}_1, \tilde{F}_1, \tilde{E}_2, \tilde{F}_2$ and the proof for relation $\mathcal{R}$. The binding property is inherited directly from the binding property of Brakedown, since the commitment phase remains unmodified from the original protocol.

What remains is to show how our PCS achieves the knowledge soundness property. We construct our extractor $\mathcal{E}$ in a way similar to the extractor proposed by Brakedown PCS [12]. Specifically, let $\mathcal{A}$ be a PPT adversary that produces an accepting transcript when interacting with the verifier over our PCS protocol. $\mathcal{E}$ rewinds $\mathcal{A}$ multiple times. Each time $\mathcal{A}$ produces an accepting transcript, $\mathcal{E}$ first constructs $\mathcal{A}_R, \mathcal{A}_{F2}$, where $\mathcal{A}_R, \mathcal{A}_{F2}$ are PPT adversaries that produce an accepting transcript when interacting with the verifier over the proof for relation $\mathcal{R}$, the polynomial commitment to $\tilde{F}_2$, respectively. Then $\mathcal{E}$ invokes $\mathcal{E}_R$ to construct $\tilde{E}_1, \tilde{F}_1, \tilde{E}_2, \tilde{F}_2$, and invokes $\mathcal{E}_{F2}$ to construct $\tilde{F}_2'$, where $\mathcal{E}_R, \mathcal{E}_{F2}$ are the extractors corresponding to $\mathcal{A}_R, \mathcal{A}_{F2}$. By the knowledge soundness of the

proof for $\mathcal{R}$ and the polynomial commitment to $\tilde{F}_2$, $\mathcal{E}$ runs in polynomial time and successfully extracts these vectors with overwhelming probability. It remains to show that, given these extracted vectors, the following conditions hold with overwhelming probability:

1. For all $j \in [\ell]$, $E_1[i_j] = \mathbf{y}_1[j]$, $E_2[i_j] = \mathbf{y}_2[j]$,and $E_1 = Enc(F_1)$, $E_2 = Enc(F_2)$ where $\mathbf{y}_1[i] = \sum_{j\in[\frac{N}{B}]} R[i,j]\mathbf{a}[j]$, $\mathbf{y}_2[i] = \sum_{j\in[\frac{N}{B}]} R[i,j]\mathsf{eq}(\mathbf{s}_1, j)$.
2. $\tilde{F}_2 = \tilde{F}_2'$ and $v = \tilde{F}_2(\mathbf{s}_2)$.

To demonstrate it, we observe that: (1)By the binding property of Merkle commitment, the probability that an adversary can find some $R' \neq R$ that produces an accepting $\mathbf{y}$ while passing verification is negligible. (2) By the binding property of the polynomial commitment to $\tilde{F}_2$, the probability that $\tilde{F}_2 \neq \tilde{F}_2'$ is negligible. Applying the union bound, we conclude that given $E_1, F_1, E_2, F_2, F_2'$, both of the above two conditions 1 and 2 hold with probability $1 - M\ell \cdot negl(\lambda)$, which is overwhelming when $M, \ell = O(1)$.

Having established that, we follow the soundness proof in Brakedown[12]. Specifically, the extractor $\mathcal{E}$ continues to rewind the adversary until it obtains $B$ linearly independent vectors $\mathbf{a}_1, \ldots, \mathbf{a}_B$ along with the corresponding reply vectors $F_{1,1}, \ldots, F_{1,B}$ that pass the verification checks of the Brakedown verifier. The extractor then applies Gaussian elimination to recover the vector $F_f \in \mathbb{F}^N$, which contains the coefficients of the polynomial $f$. In terms of performance, our extractor incurs an additional factor of $poly(\lambda, \frac{N}{B})$ increasing factor in running time compared to [12], as it executes the sub-extractor $\mathcal{E}_R$(as explained above) on each rewinding step. Since the original Brakedown extractor runs in expected polynomial time with respect to $\lambda$ and $N$, as demonstrated in [12], we conclude that $\mathcal{E}$ runs in (expected) polynomial time.

**Complexity** The complexity analysis of our distributed Brakedown(Protocol 11) is summarized as follows.

*Prover time* The main computational tasks of each sub-prover $\mathcal{P}_i$ are as follows: (1) Computing the vectors $E_1^{(i)}, F_1^{(i)}, E_2^{(i)}, F_2^{(i)}$, which requires $O(\frac{N}{B} \cdot b) = O(\frac{N}{M})$ field operations. (2) Committing to and opening $\tilde{E}_1, \tilde{F}_1, \tilde{E}_2, \tilde{F}_2$ using underlying polynomial commitment scheme $PC$, where each $\mathcal{P}_i$ incurs a cost of $t_P(\frac{N}{B})$, with $t_P$ denoting the prover time complexity of $PC$. (3)Committing to and opening vectors using the Merkle commitment scheme $MT$, where each $\mathcal{P}_i$ performs $O(\frac{N}{B})$ hash operations. (4) Computing $\tilde{H}_{\mathbf{u}}$ and $\tilde{H}_{\mathbf{u},k}^{(i)}, \tilde{E}_k^{(i)}$ during the sumcheck protocol when proving the relation $\mathcal{R}$, where each $\mathcal{P}_i$ performs $O(\frac{cN}{B})$ field operations. Overall, each sub-prover $\mathcal{P}_i$ performs $O(\frac{cN}{B})$ field operations and $O(\frac{N}{B})$ hash operations, in addition to the cost $t_P(\frac{N}{B})$ incurred by the underlying polynomial commitment scheme.

*Verifier time* The main computational tasks of $\mathcal{V}$ are as follows: (1) Verifying the proof for commitments to $\tilde{E}_1, \tilde{F}_1, \tilde{E}_2, \tilde{F}_2$, where the running time of $\mathcal{V}$ is

$M\ell t_V(\frac{N}{B})$, with $t_V$ denoting the verifier time complexity of $PC$. (2) Verifying the proof for the Merkle commitments, where $\mathcal{V}$ performs $O(M\ell \log \frac{cN}{B})$ hash operations. (3) Computing $\mathbf{y}_1, \mathbf{y}_2$ in step 10b and 10d, where $\mathcal{V}$ performs $O(B\ell)$ field operations. (4) Verifying during the sumcheck protocol when proving the relation $\mathcal{R}$, where $\mathcal{V}$ performs $O(M \log \frac{cN}{B})$ field operations. Overall, the verifier $\mathcal{V}$ performs $O(M \log \frac{cN}{B} + B\ell)$ field operations and $O(M\ell \log \frac{cN}{B})$ hash operations, in addition to the cost $\mathcal{V}$ is $M\ell t_V(\frac{N}{B})$ incurred by the underlying polynomial commitment scheme.

*Communication cost and proof size* The communication costs mainly consist of (1) the random vector $\mathbf{a} \in \mathbb{F}^B$ from $\mathcal{V}$ to $\mathcal{P}_0$, with total size $O(MB)$.(2) the proof of Merkle hash commitments, with total size $O(M\ell \log \frac{cN}{B})$. (3) the proof of commitments to $\tilde{E}_1, \tilde{F}_1, \tilde{E}_2, \tilde{F}_2$, with total size $Ms_P(\frac{N}{B})$, with $s_P$ denoting the proof size of $PC$. (4) $\tilde{E}_k^{(i)}$ from $\mathcal{P}_i$ to $\mathcal{V}$ when proving the relation $\mathcal{R}$, with total size $O(M \log \frac{cN}{B})$. The total communication size is $O(MB + M\ell \log \frac{cN}{B}) + Ms_P(\frac{N}{B})$. The proof size complexity is the same.

We assume the reciprocal of the code rate $c$ is a constant, and set $B = M \log \frac{N}{M}$. If the underlying polynomial commitment scheme $PC$ has complexity $t_P(n) = O(n \log n), t_V(n) = s_P(n) = O(\log^2 n)$, our distributed polynomial commitment achieves the following performance:

- Each prover $\mathcal{P}_i$ runs in $O(\frac{N}{M})$ time.
- The verifier $\mathcal{V}$ runs in $O(M \log^2 \frac{N}{M})$ time.
- The proof size is $O(M \log^2 \frac{N}{M})$.

We can select BaseFold [32] as the underlying $PC$, because it is transparent, post-quantum secure, and satisfies the required complexity properties. As a result, we achieve the transparent distributed polynomial commitment with linear prover time, polylogarithmic verifier time, and proof size.

## 5 Conclusion

Based on the constructions and complexity analyses presented in Section 3 and Section 4, we obtain two fully distributed SNARK schemes. Specifically,

- By compiling our distributed polynomial commitment scheme (Section 4) with our distributed PIOP for R1CS (Section 3), we achieve fully distributed SNARK for R1CS.
- By compiling our distributed polynomial commitment scheme (Section 4) with the distributed Hyperplonk PIOP from Hyperpianist [15], we achieve fully distributed SNARK for Plonkish circuit.

Both resulting distributed SNARK schemes are transparent, post-quantum secure, and feature a linear-time prover with polylogarithmic verification time and proof size.

## References


1. Ames, S., Hazay, C., Ishai, Y., Venkitasubramaniam, M.: Ligero: Lightweight sublinear arguments without a trusted setup. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. pp. 2087–2104. Association for Computing Machinery (2017)
2. Ben-Sasson, E., Chiesa, A., Riabzev, M., Spooner, N., Virza, M., Ward, N.P.: Aurora: Transparent succinct arguments for r1cs. In: Advances in Cryptology – EUROCRYPT 2019. pp. 103–128. Springer (2019)
3. Blum, M., Evans, W., Gemmell, P., Kannan, S., Naor, M.: Checking the correctness of memories. In: Proceedings of the 32nd Annual Symposium on Foundations of Computer Science (FOCS). IEEE Computer Society (1991)
4. Bootle, J., Chiesa, A., Hu, Y., Orrú, M.: Gemini: Elastic snarks for diverse environments. In: Dunkelman, O., Dziembowski, S. (eds.) Advances in Cryptology – EUROCRYPT 2022. pp. 427–457. Springer (2022)
5. Bowe, S., Hornby, T.G., Wilcox, N.: Zcash protocol specification (2016)
6. Bünz, B., Bootle, J., Boneh, D., Poelstra, A., Wuille, P., Maxwell, G.: Bulletproofs: Short proofs for confidential transactions and more. In: 2018 IEEE Symposium on Security and Privacy (SP). pp. 315–334. IEEE Computer Society (2018)
7. Chen, B., Bünz, B., Boneh, D., Zhang, Z.: Hyperplonk: Plonk with linear-time prover and high-degree custom gates. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 499–530. Springer (2023)
8. Chiesa, A., Hu, Y., Maller, M., Mishra, P., Vesely, W., Ward, N.P.: Marlin: Preprocessing zksnarks with universal and updatable srs. In: Advances in Cryptology – EUROCRYPT 2020. p. 738–768. Springer (2020)
9. Ethereum Foundation: Zk-rollups: Scaling solutions for ethereum (2024)
10. Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: Conference on the theory and application of cryptographic techniques. pp. 186–194. Springer (1986)
11. Gennaro, R., Gentry, C., Parno, B., Raykova, M.: Quadratic span programs and succinct nizks without pcps. In: Advances in Cryptology–EUROCRYPT 2013. pp. 626–645. Springer (2013)
12. Golovnev, A., Lee, J., Setty, S.T.V., Thaler, J., Wahby, R.S.: Linear-time and field-agnostic snarks for r1cs. In: Advances in Cryptology - CRYPTO 2023. pp. 193–226. Springer (2023)
13. Groth, J.: On the size of pairing-based non-interactive arguments. In: Advances in Cryptology - EUROCRYPT 2016. Lecture Notes in Computer Science, vol. 9666, pp. 305–326. Springer (2016)
14. Haböck, U.: Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive (2022)
15. Li, C., Zhu, P., Li, Y., Hong, C., Qu, W., Zhang, J.: HyperPianist: Pianist with linear-time prover and logarithmic communication cost. Cryptology ePrint Archive (2024)
16. Li, W., Zhang, Z., Chow, S.S.M., Guo, Y., Gao, B., Song, X., Deng, Y., Liu, J.: Shred-to-shine metamorphosis in polynomial commitment evolution. Cryptology ePrint Archive (2025)
17. Li, W., Zhang, Z., Li, Y., Zhu, P., Hong, C., Liu, J.: Soloist: Distributed SNARKs for rank-one constraint system. Cryptology ePrint Archive (2025)

18. Liu, T., Xie, T., Zhang, J., Song, D., Zhang, Y.: Pianist: Scalable zkrollups via fully distributed zero-knowledge proofs. In: Proceedings of the 45th IEEE Symposium on Security and Privacy (S&P '24). pp. 1777–1793. IEEE (2024)
19. Lund, C., Fortnow, L., Karloff, H., Nisan, N.: Algebraic methods for interactive proof systems. Journal of the ACM (JACM) **39**(4), 859–868 (1992)
20. Nair, V., Sharma, A., Thankey, B.: BrakingBase - a linear prover, poly-logarithmic verifier, field agnostic polynomial commitment scheme. Cryptology ePrint Archive (2025)
21. Pappas, C., Papadopoulos, D.: Hobbit: Space-efficient zksnark with optimal prover time. In: Proceedings of the 34th USENIX Security Symposium. pp. 4917–4936 (2025)
22. Rosenberg, M., Mopuri, T., Hafezi, H., Miers, I., Mishra, P.: Hekaton: Horizontally-scalable zkSNARKs via proof aggregation. Cryptology ePrint Archive (2024)
23. Setty, S.T.V.: Efficient and general-purpose zksnarks without trusted setup. In: Advances in Cryptology - CRYPTO 2020. pp. 704–737. Springer (2020)
24. Wahby, R.S., Tzialla, I., Shelat, A., Thaler, J., Walfish, M.: Doubly-efficient zk-snarks without trusted setup. In: 2018 IEEE Symposium on Security and Privacy (SP). pp. 926–943. IEEE Computer Society (2018)
25. Wang, W., Shi, F., Vilardell, D., Zhang, F.: Cirrus: Performant and accountable distributed SNARK. Cryptology ePrint Archive (2024)
26. Wu, H., Zheng, W., Chiesa, A., Popa, R.A., Stoica, I.: Dizk: A distributed zero knowledge proof system. In: Proceedings of the 27th USENIX Security Symposium (USENIX Security '18). pp. 675–692 (2018)
27. Xie, T., Zhang, J., Zhang, Y., Papamanthou, C., Song, D.: Libra: Succinct zero-knowledge proofs with optimal prover computation. In: Advances in Cryptology - CRYPTO 2019. pp. 733–764. Springer (2019)
28. Xie, T., Zhang, J., Cheng, Z., Zhang, F., Zhang, Y., Jia, Y., Boneh, D., Song, D.: zkbridge: Trustless cross-chain bridges made practical. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS 2022). pp. 3003–3017. ACM (2022)
29. Xie, T., Zhang, Y., Song, D.: Orion: Zero knowledge proof with linear prover time. In: Advances in Cryptology – CRYPTO 2022. pp. 299–328. Springer (2022)
30. Xu, H., Gama, M., Beni, E.H., Kang, J.: FRIttata: Distributed proof generation of FRI-based SNARKs. Cryptology ePrint Archive (2025)
31. Yu, Y., Liu, M., Zhang, Y., Sun, S.F., Ma, T., Au, M.H., Gu, D.: HyperFond: A transparent and post-quantum distributed SNARK with polylogarithmic communication. Cryptology ePrint Archive (2025)
32. Zeilberger, H., Chen, B., Fisch, B.: Basefold: Efficient field-agnostic polynomial commitment schemes from foldable codes pp. 138–169 (2024)

## A Sumcheck Protocol

We present the sumcheck PIOP for the relation $\mathcal{R}_{SUM}$ for high degree polynomials in [7]. Consider $f(X) := h(g_1(X), \ldots, g_c(X))$, the prover can compute the univariate polynomial $f_k$ in the $k$-th round in linear time by adapting the optimal algorithm from [27]. The construction is shown in Protocol A.1.

**Protocol A.1 Sumcheck PIOP for high degree polynomials**.
$\mathcal{P}$ claims $((v, [[f]]); f) \in \mathcal{R}_{HSUM}$ to $\mathcal{V}$ for a $\mu$-variate degree-$d$ polynomial $f$ such that $\sum_{b\in\{0,1\}^\mu} f(b) = v$.

1. For $k = 1, 2, \ldots, \mu$,
   (a) The prover computes $f_k(X) = \sum_{\mathbf{b}\in\{0,1\}^{\mu-i}} f(r_1, \ldots, r_{k-1}, X, \mathbf{b})$ and sends the oracle $[[f_k]]$ to the verifier. $f_k$ is univariate and of degree at most $d$.
   (b) The verifier checks that $v = f_k(0) + f_k(1)$, if the check fails, abort; else the verifier samples $r_k \leftarrow \mathbb{F}$, sends $r_k$ to the verifier, and sets $v \leftarrow f_k(r_k)$.
2. Finally, the verifier accepts if $f(r_1, \ldots, r_\mu) = v$.

The completeness of $\mathcal{R}_{SUM}$ is obvious. By Theorem 3.1 of [7], the sum-check protocol is proven to be knowledge sound.

Hyperpianist [15] provides distributed sumcheck PIOP, which is the distributed variant of sumcheck PIOP. The construction is shown in Protocol A.2.

**Protocol A.2 Distributed sumcheck PIOP**.
$\mathcal{P}_0$ claims $((v, [\![f]\!]); f) \in \mathcal{R}_{SUM}$ to $\mathcal{V}$ where $f \in \mathcal{F}_n^{(\leq 1)}$. $\mathcal{P}_i$ holds $f^{(i)}(x) = f(x, \text{bin}(i))$, $i \in [M]$. Let $f_0 := v$.

1. In the $k$-th round where $1 \leq k \leq n - m$:
   (a) Each $\mathcal{P}_i$ sends a local univariate polynomial $f_k^{(i)}(X_k) := \sum_{x\in\{0,1\}^{n-m-k}} f(r_1, \ldots, r_{k-1}, X_k, x, \text{bin}(i))$ to $\mathcal{P}_0$.
   (b) $\mathcal{P}_0$ sums up all the univariate polynomials to get $f_k(X_k) = \sum_{i\in[M]} f_k^{(i)}(X_k)$,
   and sends it to $\mathcal{V}$.
   (c) $\mathcal{V}$ checks if $f_{k-1} = f_k(0) + f_k(1)$. If the check passes, $\mathcal{V}$ sends a random $r_k \in \mathbb{F}$ to $\mathcal{P}_0$, and sets $f_k := f_k(r_k)$.
   (d) $\mathcal{P}_0$ transmits $r_k$ to the other $\mathcal{P}_i$. Each $\mathcal{P}_i$ then updates the local evaluations of $f^{(i)}(x)$.
2. Each $\mathcal{P}_i$ sends $f^{(i)}(r_1, \cdots, r_{n-m})$ to $\mathcal{P}_0$. $\mathcal{P}_0$ constructs the polynomial $f'(x) = f(r_1, \cdots, r_{n-m}, x)$.
3. $\mathcal{P}_0$ and $\mathcal{V}$ run the SumCheck PIOP to check $f_{n-m} = \sum_{x\in\{0,1\}^m} f'(x)$.